

Nova Developer Kit Reference

Assembly Runtime Libraries

Authored Draft

May 2026

Preface

The Nova Developer Kit Reference is the assembly programmer's companion to the Nova runtime libraries. It combines authored system chapters with a generated API reference built directly from structured comments in the runtime sources.

Nova Note

The generated reference keeps symbol names, pseudo-registers, calling conventions, and result codes close to the code that implements them. Authored chapters explain the operating model, common paths, and caveats that are hard to understand from API boxes alone.

The normal assembly pattern is to include interface files near the top of a program, call the routines you need, and include implementation files near the end of the source file.

```
.include "fio.inc"

    jsr    fio_load
    rts

.include "fio.s"
```

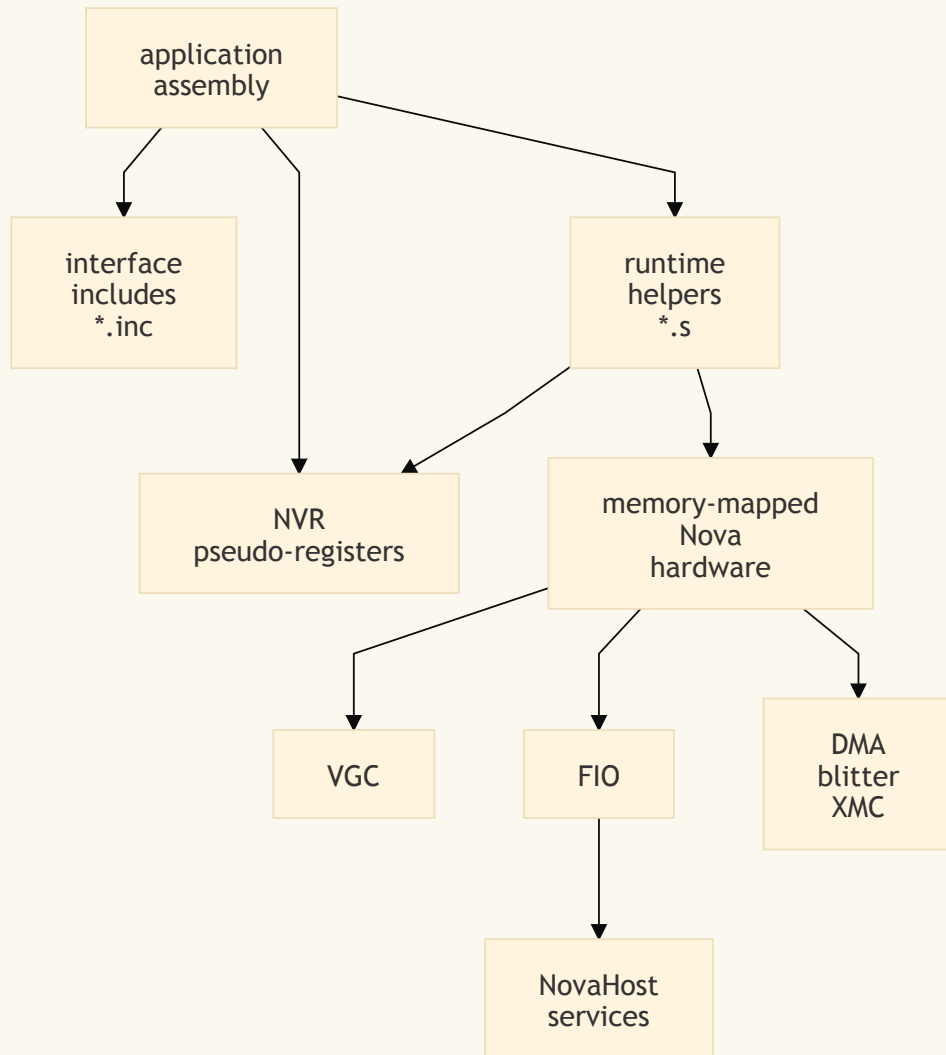
Contents

Preface	iii
1 NDK Runtime Model	1
1.1 How Libraries Are Used	1
1.2 Calling Convention	2
1.3 Hardware And Host Boundaries	2
1.4 Generated Reference	2
2 FIO Library	3
2.1 What FIO Is	3
2.2 Command Lifecycle	4
2.3 Filename Handling	4
2.4 CPU RAM Loads And Saves	5
2.5 VGC, XRAM, And Paged Transfers	5
2.6 Directory And Filesystem Commands	6
2.7 Runtime Services	7
2.8 Using FIO Well	7
3 VGC Library	9
3.1 What The VGC Is	9
3.2 Components	11
3.3 Command Helpers	11
3.4 Drawing From Assembly	12
3.5 Text And Display State	12
3.6 Off-Bus VGC Memory	13
3.7 Interrupts	14
3.8 Getting The Most From VGC	15
4 Runtime Library Reference	17
4.1 Nova Core	17
4.2 File I/O	19
4.3 Audio and Music	24
4.4 Video and Graphics Controller	34
4.5 Sprites	40
4.6 Meta-Sprites	45
4.7 Vector Sprites	49
4.8 Nova UI	58
4.9 Copper	58
4.10 DMA	60
4.11 Blitter	61
4.12 Expansion RAM	62
4.13 Expansion Memory Controller	66
4.14 File Pager	69
4.15 Random Numbers	72
4.16 Nova Graphics Images	73
4.17 Animation	74
4.18 Tweening	78

4.19 Network Interface	84
4.20 Game Server	86
4.21 Overlays	92
4.22 VGC Wait Helpers	93
About This Generated Reference	95

NDK Runtime Model

The Nova Developer Kit is a ca65 assembly library stack over Nova's memory-mapped hardware and NovaHost services. It is deliberately thin. Most helpers name registers, copy arguments into the hardware mailbox a subsystem expects, issue a command, and return a small status code.



1.1 How Libraries Are Used

Most libraries have an interface file and an implementation file. Include the `.inc` file near the top of an application so symbols and pseudo-register aliases are visible. Include or link the `.s` file once after the code that calls it.

```
.setcpu "65c02"

#include "fio.inc"
#include "vgc.inc"
```

```
main:
    JSR    fio_load
    JSR    vgc_vsync
    RTS

.include "fio.s"
.include "vgc.s"
```

Ordering matters for libraries that support size stripping. `fio.s` and `audio.s` emit only referenced public routines by default, plus the dependencies those routines need. If the implementation is included before the call sites, `ca65` has not seen the references yet and the routines can be stripped. Define `NOVA_EMIT_ALL_RUNTIME`, `FIO_EMIT_ALL_RUNTIME`, or `AUDIO_EMIT_ALL_RUNTIME` for ROM and debug builds that intentionally keep full implementations.

1.2 Calling Convention

The runtime libraries share a small convention, but individual routines can override it in their API entry:

- A return value of `A=0` means success. A return value of `A=1` usually means a generic error, with detail in a library result byte or hardware error register.
- `A`, `X`, and `Y` are scratch unless the routine documents them as outputs.
- `NVROL` through `NVR7H` are shared pseudo-register argument bytes. They are not persistent storage.
- Hardware-facing routines often leave detailed state in registers such as `FIO_STATUS`, `FIO_ERRCODE`, `XMC_STATUS`, or `VGC` pending `IRQ` registers.

Treat pseudo-registers and hardware command registers as mailboxes. Load them immediately before the call that needs them and copy out any result before the next subsystem reuses the same bytes.

1.3 Hardware And Host Boundaries

`nova.inc` is the shared map for Nova hardware addresses and command constants. `VGC`, `DMA`, `blitter`, `XMC`, `NIC`, `timer`, `math`, and `FIO` registers are all memory mapped from the CPU's point of view, but they do not all behave like RAM.

`VGC` memory spaces are off-bus display memories reached through `VGC` commands, the `VRAM` port, `FIO` graphics transfers, the pager, `DMA`, or the blitter. `XRAM` is a flat expansion memory device reached through `XMC` windows, `XRAM` helpers, `DMA`, or `FIO` streaming. `FIO` is a host-command mailbox; `NovaHost` resolves mounted devices, host files, media playback, runtime ROM loading, and random-number requests.

1.4 Generated Reference

The API chapter is generated from `@label` blocks in `runtime/asm`. If an API box is wrong, the fix belongs in the source comment next to the implementation, not in the generated LaTeX. Authored chapters in this book should explain workflows and caveats; generated entries should stay close to exact symbols, inputs, outputs, and required registers.

FIO Library

The FIO library is the assembly programmer's bridge to NovaHost storage and host-backed services. It is not a POSIX-style file API and it is not a normal 6502 disk controller. It is a small command mailbox at \$B9A0-\$B9EF that NovaHost watches, executes, and reports through status registers.

Nova Note

Use `fio.inc` for declarations and argument aliases. Include `fio.s` after the code that calls it. The implementation emits only referenced routines by default, so including it before the call sites can strip the routines you expected to link.

```
.include "fio.inc"

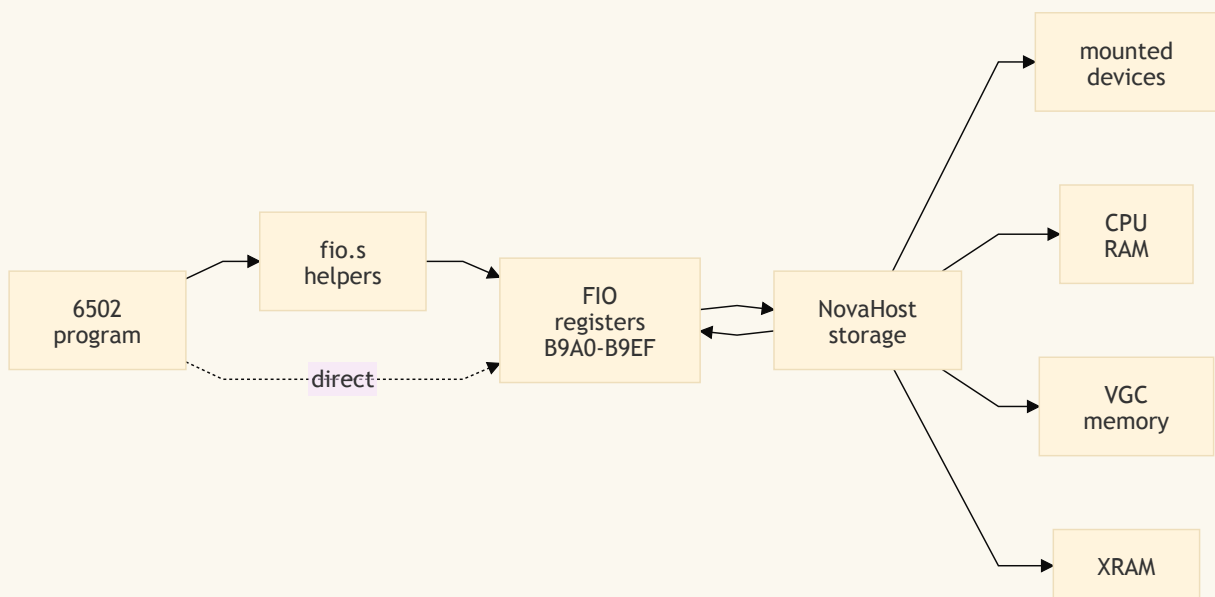
    ; program code that calls fio_load, fio_gload, etc.

.include "fio.s"
```

Define `NOVA_EMIT_ALL_RUNTIME` or `FIO_EMIT_ALL_RUNTIME` before including `fio.s` when a ROM or debugging build should keep the whole implementation.

2.1 What FIO Is

FIO is a hardware-facing command protocol plus a small set of assembly helpers. Your program fills filename, address, length, and mode registers, then issues a command. NovaHost performs the storage operation and writes back status, error, size, and directory metadata.



There are four pieces to keep straight:

Command register `FIO_CMD` triggers work when written. The library helpers use the command constants from `nova.inc`.

Status and error registers `FIO_STATUS` becomes `FIO_STATUS_OK` or `FIO_STATUS_ERROR`. On error, `FIO_ERRCODE` distinguishes not found, I/O failure, end of directory, disk full, and not mounted.

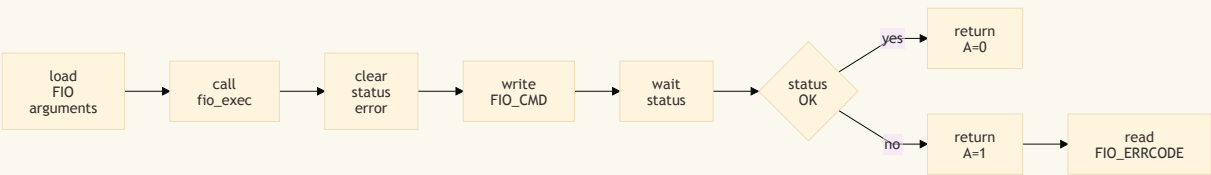
Filename buffer `FIO_NAME` holds up to 63 bytes and `FIO_NAMELEN` tells the host how many bytes are valid.

Argument registers The same register bytes are reused by different commands. For example, `FIO_SRCL/H` can be a CPU address for `fio_load`, RNG result bytes after `fio_rng`, or the low 16 bits of an `XPAGE` file offset.

This reuse is intentional. Treat FIO registers as a command mailbox, not as persistent application state.

2.2 Command Lifecycle

Most callers should use `fio_exec` or one of the named wrappers such as `fio_load`. `fio_exec` clears the previous status and error, writes the command byte, waits until the host reports completion, and returns the shared runtime result: `A` is 0 for success and 1 for error.



Helper	Use
<code>fio_issue</code>	Write <code>A</code> to <code>FIO_CMD</code> and return without waiting or checking status. Use this only when the caller owns the polling.
<code>fio_exec</code>	Issue <code>A</code> as a command, wait for completion, and return <code>A=0</code> or <code>A=1</code> .
<code>fio_check</code>	Convert the current <code>FIO_STATUS</code> value into the shared <code>A=0/A=1</code> result.
<code>fio_clear_error</code>	Clear the host-visible error latch before a retry or before returning to UI code that displays status.

The named wrappers all jump through `fio_exec`. A failed command returns `A=1`, but the useful reason is still in `FIO_ERRCODE`. Check it before preparing another FIO command, because the next command reuses the same registers.

2.3 Filename Handling

Every file command reads `FIO_NAME` and `FIO_NAMELEN`. Programs can write those registers directly, but pointer-based code should use `fio_copy_name`. It validates the length range 1..63, copies bytes from `FIO_ARG_NAMEPTR_L/H`, and returns `A=0` on success.

```

copy_level_name:
    LDA    #<level_name
    STA    FIO_ARG_NAMEPTR_L
    LDA    #>level_name
    STA    FIO_ARG_NAMEPTR_H
    LDA    #level_name_end - level_name
    STA    FIO_ARG_NAMELEN
  
```

```

        JSR    fio_copy_name
        RTS

level_name:
        .byte "LEVEL1"
level_name_end:

```

The host accepts simple host filenames and mounted-device paths. Explicit extensions matter for data files: if no extension is supplied, RAM program loads look for `.bas` and then `.bin`; graphics transfers use `.gfx`; XRAM transfers use `.xram`; paged slices use `.bin`.

2.4 CPU RAM Loads And Saves

`fio_load` moves a `.bas` or `.bin` file into CPU RAM. For `.bas`, the destination comes from `FIO_SRCL/H`. For `.bin`, the file's two-byte load-address prefix supplies the destination and NovaHost writes that address back to `FIO_SRCL/H`. The transferred payload size is returned in `FIO_SIZE/H`.

```

load_level:
        LDA    #<level_name
        STA    FIO_ARG_NAMEPTR_L
        LDA    #>level_name
        STA    FIO_ARG_NAMEPTR_H
        LDA    #level_name_end - level_name
        STA    FIO_ARG_NAMELEN
        JSR    fio_copy_name
        BNE    @error

        LDA    #<$6000
        STA    FIO_SRCL
        LDA    #>$6000
        STA    FIO_SRCH
        JSR    fio_load
        BNE    @error
        RTS

@error:
        LDA    FIO_ERRCODE
        RTS

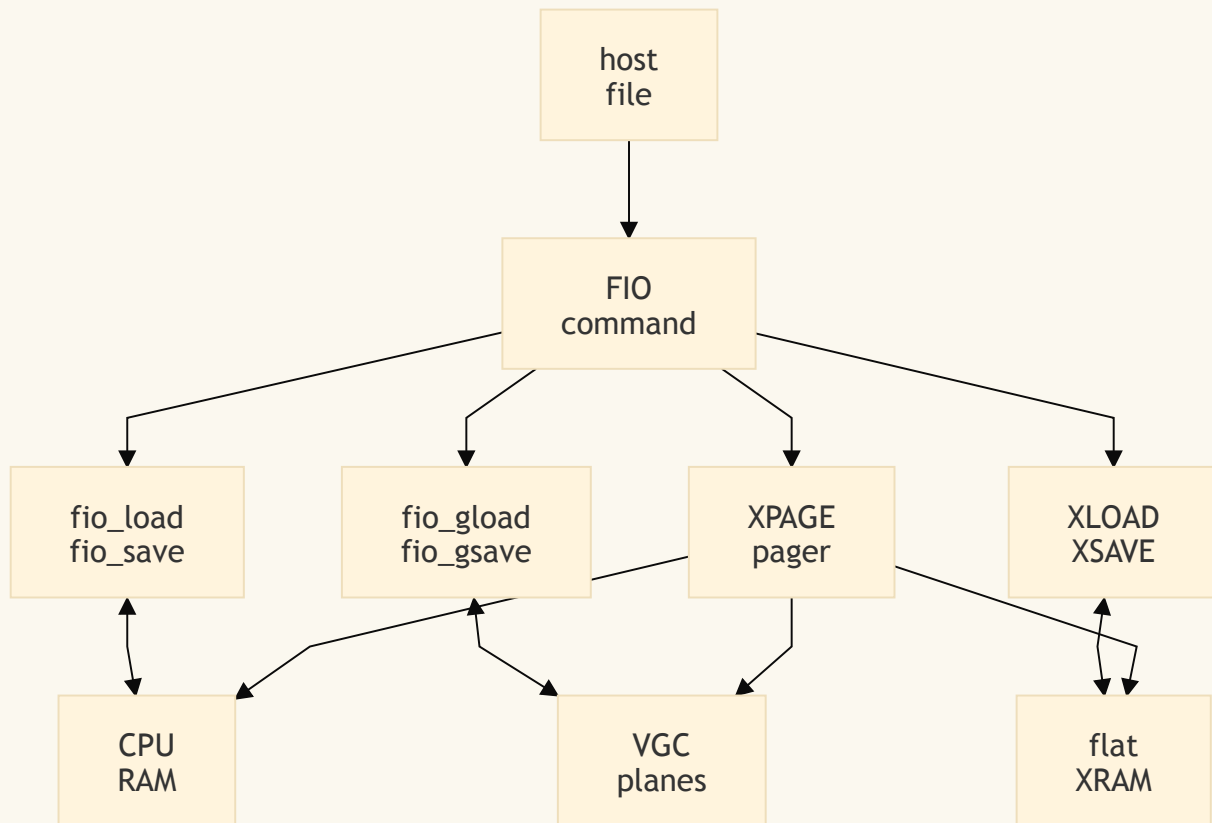
level_name:
        .byte "LEVEL1"
level_name_end:

```

`fio_save` uses `FIO_SRCL/H` as the first CPU address and `FIO_ENDL/H` as the exclusive end address. NovaHost writes a two-byte load-address prefix before the saved bytes. A default `.bas` save still loads to the caller-supplied `FIO_SRCL/H`; an explicitly saved `.bin` file uses the prefix as its load address.

2.5 VGC, XRAM, And Paged Transfers

Real Nova applications should avoid byte-copying large assets through the CPU when a bulk path exists. FIO can load directly into VGC memory, stream whole files to XRAM, or page slices of a file into XRAM, CPU RAM, or VGC memory.



Path	When to use it
<code>fio_gload</code> , <code>fio_gsave</code>	Bulk load or save VGC memory. Set <code>FIO_GSPACE</code> , <code>FIO_GADDRL/H</code> , and <code>FIO_GLENL/H</code> . A zero load length means the whole file.
<code>FIO_CMD_XLOAD</code> , <code>FIO_CMD_XSAVE</code>	Stream a file to or from flat XRAM. Use <code>fio_prepare_xram_transfer</code> to copy the pointer filename, 24-bit XRAM address, and length into FIO registers, then call <code>fio_run</code> . XLOAD accepts length zero for the whole file; XSAVE requires a nonzero length.
<code>FIO_CMD_XPAGE</code>	Load a slice of a <code>.bin</code> file. The pager library wraps this as <code>pager_load_file_page</code> ; target 0 is XRAM, target 1 is CPU RAM, and target 2 is VGC memory.

For VGC transfers, `FIO_GSPACE` is the VGC memory space and `FIO_GADDRL/H` is the 16-bit offset inside that space. For XRAM transfers, the same bytes become a 24-bit address: `FIO_GADDRL`, `FIO_GADDRH`, then `FIO_GSPACE`.

2.6 Directory And Filesystem Commands

`fio_dir_open` opens a listing and populates the first entry. `fio_dir_read` advances to the next entry. When there are no more entries, the host reports `FIO_STATUS_ERROR` with `FIO_ERR_EOD`; that is normal iteration termination, not a broken device.

Result register	Directory meaning
<code>FIO_DIRTYPE</code>	Entry type: 0 for BASIC/default, 1 for SID, 2 for BIN, 3 for MID, and 5 for directory.
<code>FIO_NAME</code> , <code>FIO_NAMELEN</code>	Display name for the current entry. Some file types append useful load metadata to the display string.
<code>FIO_SIZE2/H</code> , <code>FIO_SIZE2</code>	24-bit data size for the entry. BASIC and BIN sizes exclude the two-byte load-address prefix.

`fio_pwd`, `fio_cd`, `fio_mkdir`, and `fio_rmdir` use the same filename buffer. `fio_delete` routes through the host's program/file delete path; on hosted BASIC program files it also removes the companion `.md` document when present. Host directory devices allow paths and mounted-device prefixes; invalid names or missing mounted media report through the normal error registers.

2.7 Runtime Services

`fio_load_runtime` loads a 16K runtime ROM image into the primary \$C000 ROM bank. Code issuing this command must be running from RAM, because the host can replace the ROM bank while the command is in progress.

`fio_rng` asks NovaHost for 32 random bits and returns them in `FIO_RNG0` through `FIO_RNG3`. These aliases overlap `FIO_SRCL/H` and `FIO_ENDL/H`, so consume the random value before preparing another command.

2.8 Using FIO Well

- Always branch on the A result from the helper, then read `FIO_ERRCODE` before issuing another command.
- Use `fio_copy_name` for pointer-based filenames; it catches the empty and too-long cases before the host sees them.
- Use the bulk paths for assets. Load graphics into VGC memory with `fio_gload`, stage large data in XRAM with `XLOAD`, and use the pager when only one slice of a larger file is needed.
- Do not assume `FIO_NAME` survives across subsystems. Directory reads, current-directory queries, and other host services all reuse the same buffer.
- Clear stale host-visible errors with `fio_clear_error` before returning to code that polls FIO status independently.

VGC Library

The VGC library is the assembly programmer's thin layer over Nova's Virtual Graphics Controller. It does not hide the hardware model. It gives names to the registers, exposes the command protocol, and provides small helper routines for the operations that almost every graphics program needs.

Nova Note

Use `vgc.inc` for declarations and constants. Include `vgc.s` when the program wants the helper routines, or include `vgc_wait.s` when an overlay only needs the command wait primitive.

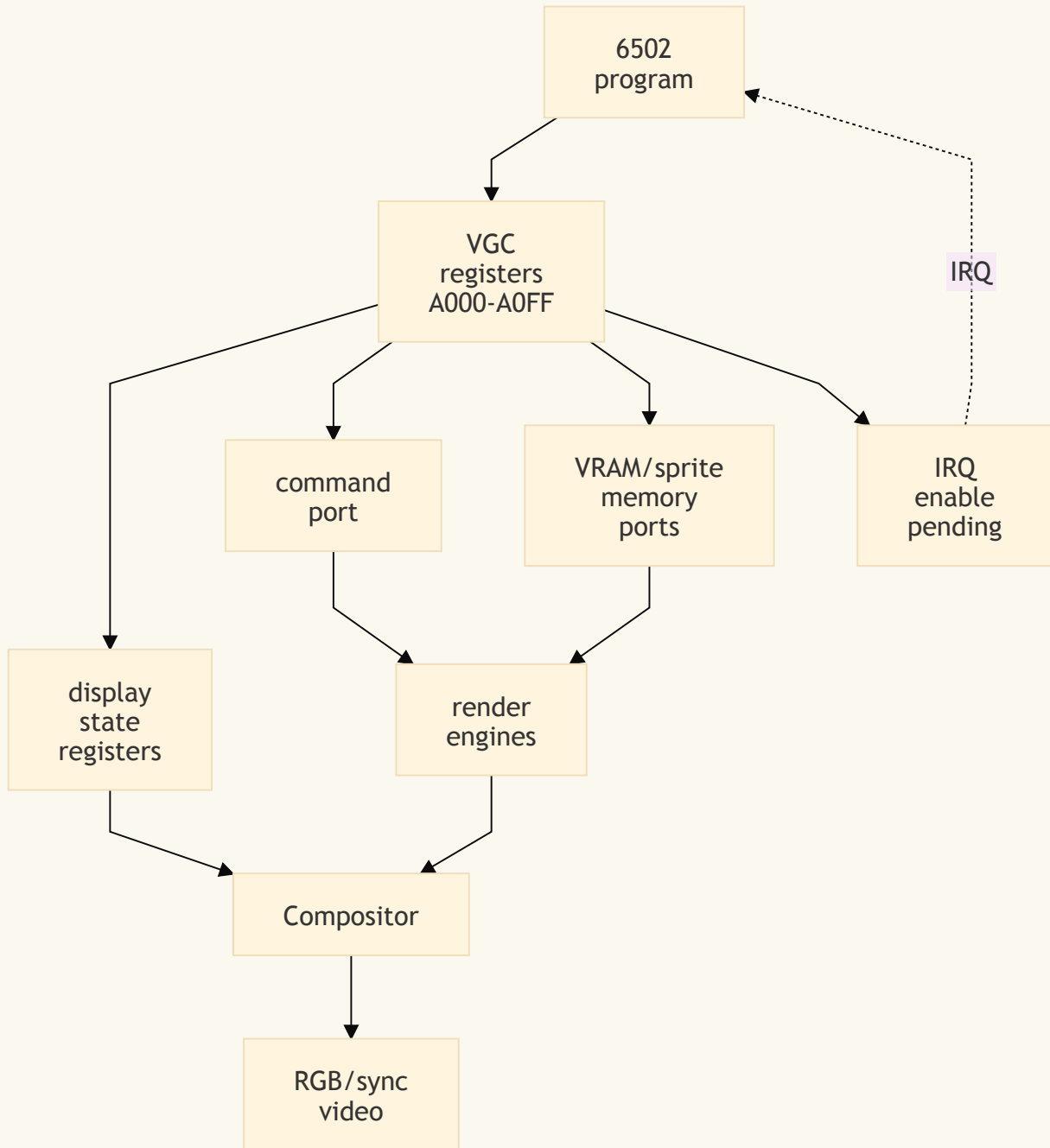
```
.include "vgc.inc"

    ; program code

.include "vgc.s"
```

3.1 What The VGC Is

The VGC is Nova's display custom chip. From the 6502 side it is a memory mapped device at \$A000-\$A0FF. Inside the device are separate engines for video timing, text, bitmap graphics, sprites, raster-time register writes, IRQs, and off-bus video memory.



The important mental model is that the CPU does not draw pixels directly into a linear framebuffer in main RAM. Instead, code talks to the VGC through three paths:

- Direct registers for state such as mode, color, cursor position, frame counter, sprite attributes, palette mode, and IRQ latches.
- The command port at \$A010–\$A01F for drawing primitives, sprite shape commands, copper list operations, and byte-level memory commands.
- The VGC memory port, DMA, blitter, FIO, and pager paths for bulk access to character RAM, color RAM, the graphics bitmap, sprite shape RAM, and text attribute RAM.

3.2 Components

VESTA timing owns the scan position, frame counter, vblank boundary, and video sync. The `vgc_vsync` helper waits for `VGC_FRAME` at `$A008` to change.

SCRIBE text renders the 80 by 50 character layer from character, color, font, and text-attribute memory. `VGC_CHAROUT` accepts printable text and control bytes such as form feed, carriage return, line feed, home, and backspace.

CANVAS graphics stores the 320 by 200, 16-color bitmap. Drawing commands such as `vgc_line`, `vgc_rect`, and `vgc_paint` operate on this memory through the ARTIST engine.

ARTIST drawing executes graphics commands from the command port. It uses the current graphics color set by `VGC.GCOLOR` and clips drawing to the bitmap.

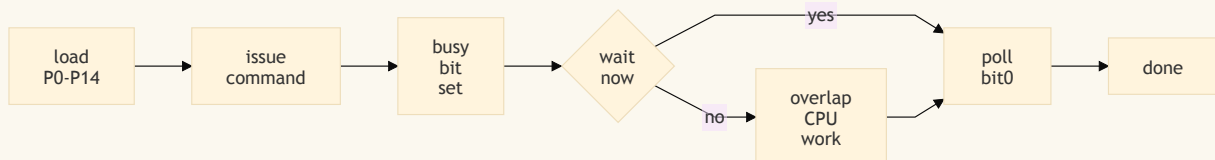
PIXIE sprites renders 16 hardware sprites. Each sprite has an 8-byte attribute block at `$A040-$A0BF`; shape pixels live in off-bus sprite shape RAM.

CONDUCTOR copper stores raster events and writes selected VGC or sprite registers when scanout reaches the programmed position.

IRQ block exposes enable, pending, force, and valid-source registers at `$A0F0-$A0F3`. The VGC can raise IRQs for vblank, copper events, and sprite collision sources.

3.3 Command Helpers

The command protocol is always the same: load `VGC_P0` through `VGC_P14`, then write a command byte to `VGC_CMD`. The helper routines in `vgc.s` save you from remembering the command byte. They do not all wait for completion.



Helper pattern	Use
<code>vgc_cmd</code>	Write A to <code>VGC_CMD</code> and return immediately.
<code>vgc_exec</code>	Write A to <code>VGC_CMD</code> , then call <code>vgc_wait_cmd</code> .
<code>vgc_wait_cmd</code>	Poll bit 0 of <code>VGC_CMD</code> until the active command completes.
<code>vgc_mem_read</code> , <code>vgc_mem_write</code>	Issue the memory command and wait, because the caller needs the byte result or address update.
<code>vgc_gcls</code> , <code>vgc_line</code> , <code>vgc_fill</code> , and other primitive helpers	Issue the drawing command and return. Wait explicitly when the next step depends on the result or reuses command parameters.

This distinction matters most on FPGA builds, where a clear, flood fill, sprite clear, sprite copy, or text clear can span many pixel clocks. Avalonia's hosted VGC often completes commands immediately, but portable code should synchronize when it depends on command completion.

3.4 Drawing From Assembly

The following example selects graphics/sprite mode, clears the bitmap, sets a draw color, and draws a diagonal line. It waits after the clear and after the line because both operations modify the graphics plane.

```
.include "vgc.inc"

draw_demo:
    LDX    #$03
    JSR    vgc_set_mode

    JSR    vgc_gcls
    JSR    vgc_wait_cmd

    LDA    #$07
    STA    VGC_P0
    JSR    vgc_gcolor

    ; LINE 0,0,319,199
    STZ    VGC_P0      ; x0 low
    STZ    VGC_P1      ; x0 high
    STZ    VGC_P2      ; y0 low
    STZ    VGC_P3      ; y0 high
    LDA    #$3F        ; x1 = $013F
    STA    VGC_P4
    LDA    #$01
    STA    VGC_P5
    LDA    #$C7        ; y1 = 199
    STA    VGC_P6
    STZ    VGC_P7
    JSR    vgc_line
    JSR    vgc_wait_cmd
    RTS

.include "vgc.s"
```

3.5 Text And Display State

The text helpers are direct register writes and return immediately: `vgc_cls`, `vgc_locate`, `vgc_set_fg`, `vgc_set_bg`, `vgc_set_border`, `vgc_set_mode`, `vgc_set_font`, `vgc_display_on`, and `vgc_display_off`.

`vgc_cls` writes form feed to `VGC_CHAROUT`; it clears the text plane and homes the cursor. `vgc_locate` copies `VGC_P0` and `VGC_P1` into the cursor X and Y registers. Foreground, background, and border helpers take their color in X.

```
    LDX    #15
    JSR    vgc_set_fg
    LDX    #0
    JSR    vgc_set_bg
    LDX    #11
    JSR    vgc_set_border

    LDA    #10
    STA    VGC_P0
```

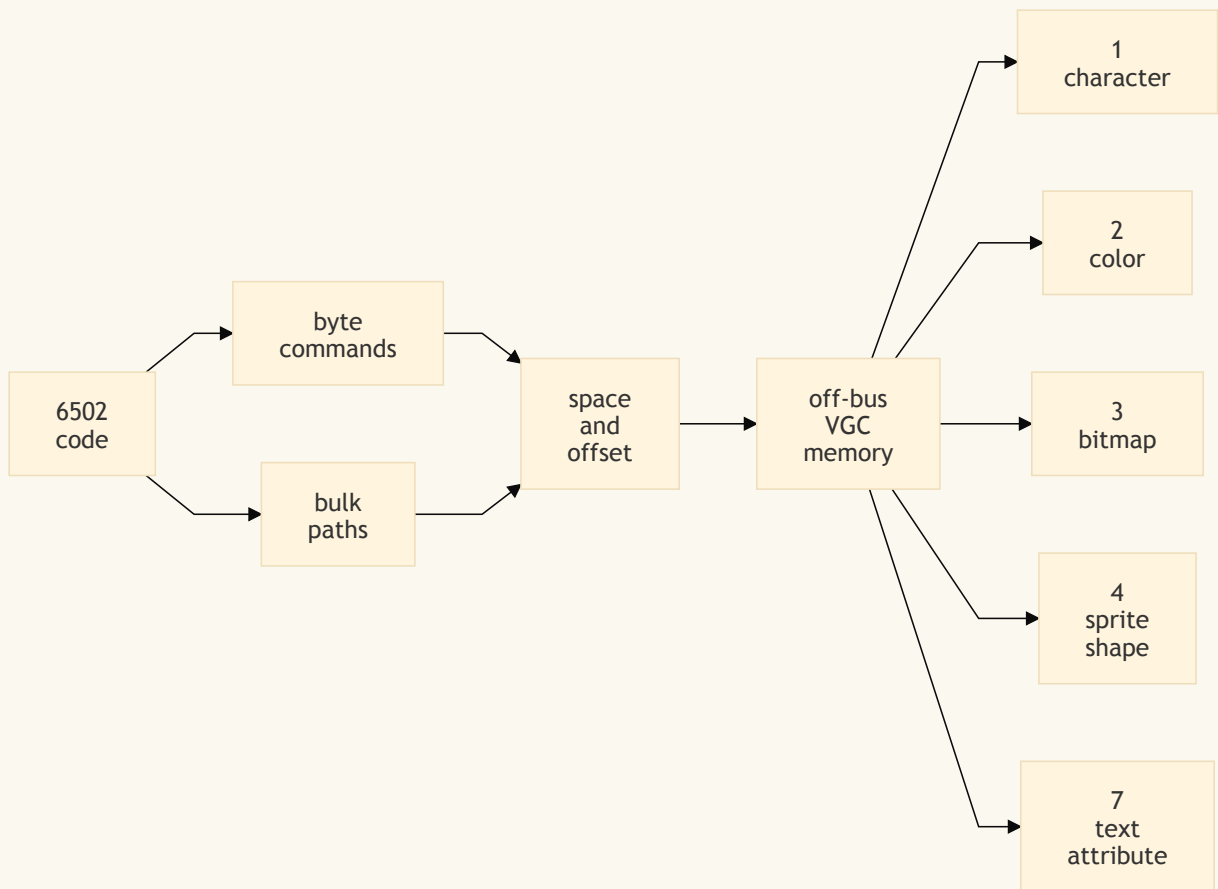
```

LDA    #6
STA    VGC_P1
JSR    vgc_locate

```

3.6 Off-Bus VGC Memory

Character RAM, color RAM, text attributes, graphics pixels, and sprite shapes are VGC memory spaces. They are not mapped into the 6502 address range as normal RAM. The library exposes `vgc_mem_read` and `vgc_mem_write` for byte-level access, while larger transfers should use the VRAM port, DMA, blitter, FIO graphics load/save, or pager routines.



Space	Name	Use
1	Character RAM	4000 character bytes for the 80 by 50 text layer.
2	Color RAM	4000 packed background/foreground color bytes.
3	Graphics bitmap	64000 pixels, one byte per pixel, low nibble is color.
4	Sprite shape RAM	32768 bytes, 256 shapes at 128 bytes per shape.
7	Text attribute RAM	4000 style bytes; bit 0 is flash.

For isolated bytes, load P0=space, P1/P2=offset, P3=data, and P4=flags. Bit 0 of P4 auto-increments the offset after each read or write.

```

; Write "OK" into character RAM at cell 0 using auto-increment.
LDA    #VGC_PLANE_CHAR
STA    VGC_P0
STZ    VGC_P1
STZ    VGC_P2

```

```

LDA    #VGC_VRAM_AUTOINC
STA    VGC_P4

LDA    #'0'
STA    VGC_P3
JSR    vgc_mem_write

LDA    #'K'
STA    VGC_P3
JSR    vgc_mem_write

```

For streams, prefer the VRAM data port. It uses the same space IDs and advances the address in hardware when `VGC_VCTRL` has `VGC_VRAM_AUTOINC` set. Reads are latched: the first read from `VGC_VDATA` posts the selected byte and the second read returns it, so read loops should prime the latch before consuming data.

```

LDA    #VGC_PLANE_GFX
STA    VGC_VPLANE
STZ    VGC_VADDRL
STZ    VGC_VADDRH
LDA    #VGC_VRAM_AUTOINC
STA    VGC_VCTRL

; Stream pixels by writing VGC_VDATA.
LDA    #$05
STA    VGC_VDATA
STA    VGC_VDATA
STA    VGC_VDATA

```

3.7 Interrupts

The VGC IRQ helpers install the CPU IRQ vector, enable or disable source masks, and acknowledge pending source bits. The source constants come from `nova.inc`; the common masks are `VGC_IRQ_VBLANK`, `VGC_IRQ_COPPER0`, `VGC_IRQ_SPRCOLL`, and `VGC_IRQ_SPRBG`.

```

install_vblank_irq:
    LDA    #>vblank_irq
    LDY    #<vblank_irq
    JSR    vgc_irq_install

    LDA    #VGC_IRQ_VBLANK
    JSR    vgc_irq_enable
    RTS

vblank_irq:
    PHA
    LDA    VGC_IRQ_STATUS
    AND    #VGC_IRQ_VBLANK
    BEQ    @done

    ; Do one-frame work here.

    LDA    #VGC_IRQ_VBLANK
    JSR    vgc_irq_ack

@done:

```

PLA
RTI

Real handlers should preserve any registers they use and must acknowledge every VGC source they service before returning. If a source remains pending and enabled, the IRQ line remains asserted.

3.8 Getting The Most From VGC

Use `vgc_vsync` as the frame boundary for animation. Update game state, write sprite attributes, and start large drawing work in a consistent order each frame.

Use direct sprite registers for frequent movement. The command helpers are convenient, but `$A040 + sprite * 8` is the faster path for X, Y, shape, flags, priority, and transparent color.

Use the graphics primitives for small generated shapes and diagnostics. Use DMA, blitter, NVG loading, or pager transfers for large images, sprite banks, and full-screen changes.

Set `VGC_GFXTRANS` deliberately. It defaults to color 0 so black graphics pixels are transparent. If a program needs visible black pixels in the graphics layer, move the transparent color to a palette index that the art does not use and keep blitter color keys in sync with that choice.

Use the copper for scanline work instead of tight CPU timing loops. Copper events are tied to the video beam and continue to run while the 6502 works on the next frame.

Runtime Library Reference

This chapter is generated from structured comments in the Nova runtime assembly sources. It lists the callable routines, constants, pseudo-registers, and shared data blocks exposed to assembly programs.

4.1 Nova Core

Generated from nova.inc.

FIO.CMD_LOADRUNTIME (const)

Symbol: FIO_CMD_LOADRUNTIME

Summary: Host command that streams a 16K file into the primary runtime ROM bank. Callers must execute from RAM while issuing it.

Value: \$28 (\$28)

Source: nova.inc line 1073

FIO.CMD_NVGLOAD (const)

Symbol: FIO_CMD_NVGLOAD

Summary: Host command that decodes a Nova NVG1 file into the graphics bitmap plane.

Value: \$2B (\$2B)

Source: nova.inc line 1094

FIO.CMD_RNG (const)

Symbol: FIO_CMD_RNG

Summary: Host command that returns 32 bits from NovaHost's host-backed RNG provider.

Value: \$2A (\$2A)

Source: nova.inc line 1087

FIO.CMD_XPAGE (const)

Symbol: FIO_CMD_XPAGE

Summary: Host command that streams a file slice into XRAM, CPU RAM, or VGC memory using FIO_DIRTYPE as the target selector.

Value: \$29 (\$29)

Source: nova.inc line 1080

FIO.NAME (buffer)

Symbol: FIO_NAME

Summary: Filename buffer used by file-backed runtime routines.

Value: \$B9B0 (\$B9B0)

Source: nova.inc line 758

FIO.NAMELEN (u8)**Symbol:** FIO_NAMELEN**Summary:** Length of the filename in FIO.NAME, 1..63 bytes.**Value:** \$B9A3 (\$B9A3)**Source:** nova.inc line 712**FIO.RNG0** (u8)**Symbol:** FIO_RNG0**Summary:** Low byte returned by FIO.CMD_RNG.**Value:** FIO_SRCL (\$B9A4)**Source:** nova.inc line 1102**FIO.RNG1** (u8)**Symbol:** FIO_RNG1**Summary:** Second byte returned by FIO.CMD_RNG.**Value:** FIO_SRCH (\$B9A5)**Source:** nova.inc line 1109**FIO.RNG2** (u8)**Symbol:** FIO_RNG2**Summary:** Third byte returned by FIO.CMD_RNG.**Value:** FIO_ENDL (\$B9A6)**Source:** nova.inc line 1116**FIO.RNG3** (u8)**Symbol:** FIO_RNG3**Summary:** High byte returned by FIO.CMD_RNG.**Value:** FIO_ENDH (\$B9A7)**Source:** nova.inc line 1123**ROMSWAP.PRIMARY** (const)**Symbol:** ROMSWAP_PRIMARY**Summary:** Select the primary runtime ROM bank at \$C000. At cold boot this is NovaBASIC; a RAM launcher may replace it with another runtime first.**Value:** ROMSWAP_BASIC (\$02)**Source:** nova.inc line 247**SYS.REGA** (u8)**Symbol:** SYS_REGA**Summary:** Returned A register from the most recent SYS call.**Value:** \$E7 (\$E7)**Source:** nova.inc line 64

SYS.REGX (u8)**Symbol:** SYS_REGX**Summary:** Returned X register from the most recent SYS call.**Value:** \$E8 (\$E8)**Source:** nova.inc line 71**SYS.REGY** (u8)**Symbol:** SYS_REGY**Summary:** Returned Y register from the most recent SYS call.**Value:** \$E9 (\$E9)**Source:** nova.inc line 78**VGC.GFXTRANS** (const)**Symbol:** VGC_GFXTRANS**Summary:** Graphics-plane transparent color register. Defaults to 0; set to another palette index when graphics code needs visible palette-0 black pixels.**Value:** \$A0E8 (\$A0E8)**Source:** nova.inc line 363

4.2 File I/O

*Generated from fio.inc, fio.s.***FIO.ARG_ADDRH** (zp)**Symbol:** FIO_ARG_ADDRH**Summary:** High byte of XRAM address argument for FIO.PREPARE_XRAM_TRANSFER.**Value:** NVR1L (\$22)**Source:** fio.inc line 34**FIO.ARG_ADDRL** (zp)**Symbol:** FIO_ARG_ADDRL**Summary:** Low byte of XRAM address argument for FIO.PREPARE_XRAM_TRANSFER.**Value:** NVR0L (\$20)**Source:** fio.inc line 24**FIO.ARG_ADDRM** (zp)**Symbol:** FIO_ARG_ADDRM**Summary:** Middle byte of XRAM address argument for FIO.PREPARE_XRAM_TRANSFER.**Value:** NVR0H (\$21)**Source:** fio.inc line 29

FIO.ARG_LENH (zp)**Symbol:** FIO_ARG_LENH**Summary:** High byte of transfer length for FIO.PREPARE_XRAM_TRANSFER.**Value:** NVR3H (\$27)**Source:** fio.inc line 44**FIO.ARG_LENL** (zp)**Symbol:** FIO_ARG_LENL**Summary:** Low byte of transfer length for FIO.PREPARE_XRAM_TRANSFER.**Value:** NVR3L (\$26)**Source:** fio.inc line 39**FIO.ARG_NAMELEN** (zp)**Symbol:** FIO_ARG_NAMELEN**Summary:** Filename length for FIO.COPY_NAME.**Value:** NVR4L (\$28)**Source:** fio.inc line 49**FIO.ARG_NAMEPTR_H** (zp)**Symbol:** FIO_ARG_NAMEPTR_H**Summary:** High byte of filename pointer for FIO.COPY_NAME.**Value:** NVR5L (\$2A)**Source:** fio.inc line 59**FIO.ARG_NAMEPTR_L** (zp)**Symbol:** FIO_ARG_NAMEPTR_L**Summary:** Low byte of filename pointer for FIO.COPY_NAME.**Value:** NVR4H (\$29)**Source:** fio.inc line 54**FIO.CD** (routine)**Symbol:** fio_cd**Summary:** Change directory to FIO.NAME/FIO.NAMELEN.**Requires:** FIO_NAME, FIO_NAMELEN**Source:** fio.s line 303**FIO.CHECK** (routine)**Symbol:** fio_check**Summary:** Convert the current FIO status register to the shared A=0/1 result.**Outputs:**

- A: 0 on success, 1 on error.

Source: fio.s line 129

FIO.CLEAR_ERROR (routine)**Symbol:** `fio_clear_error`**Summary:** Clear the host-visible FIO status and error latch.**Outputs:**

- A: 0 on success.

Source: `fio.s` line 167**FIO.COPY_NAME** (routine)**Symbol:** `fio_copy_name`**Summary:** Copy a pointer-based filename into FIO.NAME/FIO.NAMELEN.**Outputs:**

- A: 0 on success, 1 on invalid name.

Requires: FIO_ARG_NAMELEN, FIO_ARG_NAMEPTR_L, FIO_ARG_NAMEPTR_H**Source:** `fio.s` line 201**FIO.DELETE** (routine)**Symbol:** `fio_delete`**Summary:** Delete the host file or BASIC program named by FIO.NAME/FIO.NAMELEN.**Requires:** FIO_NAME, FIO_NAMELEN**Source:** `fio.s` line 270**FIO.DIR_OPEN** (routine)**Symbol:** `fio_dir_open`**Summary:** Open a directory listing using optional FIO.NAME/FIO.NAMELEN filter.**Source:** `fio.s` line 250**FIO.DIR_READ** (routine)**Symbol:** `fio_dir_read`**Summary:** Read the next directory entry into the FIO directory result registers.**Source:** `fio.s` line 260**FIO.EXEC** (routine)**Symbol:** `fio_exec`**Summary:** Issue an FIO command, wait for completion, and return A=0/1 status.**Inputs:**

- A: FIO command byte.

Outputs:

- A: 0 on success, 1 on error.

Source: `fio.s` line 147

FIO.GLOAD (routine)**Symbol:** `fio_gload`**Summary:** Load VGC/graphics memory using FIO graphics registers.**Requires:** `FIO_NAME`, `FIO_NAMELEN`, `FIO_GSPACE`, `FIO_GADDRL`, `FIO_GADDRH`, `FIO_GLENL`, `FIO_GLENH`**Source:** `fio.s` line 292**FIO.GSAVE** (routine)**Symbol:** `fio_gsave`**Summary:** Save VGC/graphics memory using FIO graphics registers.**Requires:** `FIO_NAME`, `FIO_NAMELEN`, `FIO_GSPACE`, `FIO_GADDRL`, `FIO_GADDRH`, `FIO_GLENL`, `FIO_GLENH`**Source:** `fio.s` line 281**FIO.ISSUE** (routine)**Symbol:** `fio_issue`**Summary:** Issue a raw FIO command without waiting for completion.**Inputs:**

- A: FIO command byte.

Source: `fio.s` line 117**FIO.LOAD** (routine)**Symbol:** `fio_load`**Summary:** Load a BASIC or binary file using `FIO.NAME` and `FIO_SRCL/H`.**Requires:** `FIO_NAME`, `FIO_NAMELEN`, `FIO_SRCL`, `FIO_SRCH`**Source:** `fio.s` line 239**FIO.LOAD_RUNTIME** (routine)**Symbol:** `fio_load_runtime`**Summary:** Load a 16K runtime ROM image named by `FIO.NAME`/`FIO.NAMELEN` into the primary runtime ROM bank.**Requires:** `FIO_NAME`, `FIO_NAMELEN`**Source:** `fio.s` line 349**FIO.MKDIR** (routine)**Symbol:** `fio_mkdir`**Summary:** Create the directory named by `FIO.NAME`/`FIO.NAMELEN`.**Requires:** `FIO_NAME`, `FIO_NAMELEN`**Source:** `fio.s` line 314**FIO.PREPARE_XRAM_TRANSFER** (routine)**Symbol:** `fio_prepare_xram_transfer`**Summary:** Copy pointer filename and XRAM transfer arguments into FIO registers.

Requires: FIO_ARG_NAMELEN, FIO_ARG_NAMEPTR_L, FIO_ARG_NAMEPTR_H, FIO_ARG_ADDRL, FIO_ARG_ADDRM, FIO_ARG_ADDRH, FIO_ARG_LENL, FIO_ARG_LENH
Source: fio.s line 385

FIO.PWD (routine)

Symbol: fio_pwd
Summary: Read the current directory into FIO.NAME/FIO.NAMELEN.
Source: fio.s line 336

FIO.RMDIR (routine)

Symbol: fio_rmdir
Summary: Remove the directory named by FIO.NAME/FIO_NAMELEN.
Requires: FIO_NAME, FIO_NAMELEN
Source: fio.s line 325

FIO.RNG (routine)

Symbol: fio_rng
Summary: Issue FIO.CMD_RNG and return random bytes in FIO_RNG0..3.
Outputs:

- A: 0 on success, 1 on error.
- FIO_RNG0: FIO_RNG1 FIO_RNG2 FIO_RNG3

Source: fio.s line 361

FIO.RUN (routine)

Symbol: fio_run
Summary: Compatibility alias for FIO.EXEC.
Source: fio.s line 375

FIO.SAVE (routine)

Symbol: fio_save
Summary: Save RAM using FIO.NAME plus FIO_SRCL/H and FIO_ENDL/H.
Requires: FIO_NAME, FIO_NAMELEN, FIO_SRCL, FIO_SRCH, FIO_ENDL, FIO_ENDH
Source: fio.s line 228

FIO.SET_IO_ERROR (routine)

Symbol: fio_set_io_error
Summary: Set FIO_STATUS/FIO_ERRCODE to the shared I/O error state.
Outputs:

- A: 1 on error.

Source: fio.s line 185

4.3 Audio and Music

Generated from *audio.inc*, *audio.s*.

AUDIO.ATTACK (u8)

Symbol: AUDIO_ATTACK

Summary: Attack nibble for AUDIO.INSTRUMENT_SET.

Value: F10_ENDL (\$B9A6)

Source: audio.inc line 226

AUDIO.DECAY (u8)

Symbol: AUDIO_DECAY

Summary: Decay nibble for AUDIO.INSTRUMENT_SET.

Value: F10_ENDH (\$B9A7)

Source: audio.inc line 231

AUDIO.DURATION (u8)

Symbol: AUDIO_DURATION

Summary: Duration in 60 Hz frames for AUDIO.SOUND.

Value: F10_SRCH (\$B9A5)

Source: audio.inc line 194

AUDIO.INIT (routine)

Symbol: audio_init

Summary: Initialize the SID sound-effect instrument table and scheduler state.

ABI: none

Source: audio.s line 356

AUDIO.INSTRUMENT (u8)

Symbol: AUDIO_INSTRUMENT

Summary: Instrument slot for AUDIO.SOUND.

Value: F10_ENDL (\$B9A6)

Source: audio.inc line 199

AUDIO.INSTRUMENT_SET (routine)

Symbol: audio_instrument

Summary: Define a simple SID sound-effect instrument envelope and waveform.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: AUDIO.INST_ID, AUDIO.WAVEFORM, AUDIO.ATTACK, AUDIO.DECAY, AUDIO.SUSTAIN, AUDIO.RELEASE

Source: audio.s line 310

AUDIO.INST_ID (u8)**Symbol:** AUDIO_INST_ID**Summary:** Instrument slot to define for AUDIO.INSTRUMENT_SET.**Value:** F10_SRCL (\$B9A4)**Source:** audio.inc line 216**AUDIO.META_AUTHOR (bytes)****Symbol:** AUDIO_META_AUTHOR**Summary:** 32-byte null-padded active music author/composer.**Value:** \$BAD3 (\$BAD3)**Source:** audio.inc line 108**AUDIO.META_COPYRIGHT (bytes)****Symbol:** AUDIO_META_COPYRIGHT**Summary:** 32-byte null-padded active music copyright/comment.**Value:** \$BAF3 (\$BAF3)**Source:** audio.inc line 113**AUDIO.META_DURATION_H (u8)****Symbol:** AUDIO_META_DURATION_H**Summary:** Duration seconds high byte when known.**Value:** \$BB1B (\$BB1B)**Source:** audio.inc line 158**AUDIO.META_DURATION_L (u8)****Symbol:** AUDIO_META_DURATION_L**Summary:** Duration seconds low byte when known.**Value:** \$BB1A (\$BB1A)**Source:** audio.inc line 153**AUDIO.META_FLAGS (u8)****Symbol:** AUDIO_META_FLAGS**Summary:** Music-specific flags for the active metadata block.**Value:** \$BB1C (\$BB1C)**Source:** audio.inc line 163**AUDIO.META_INIT_H (u8)****Symbol:** AUDIO_META_INIT_H**Summary:** SID init address high byte when applicable.**Value:** \$BB16 (\$BB16)**Source:** audio.inc line 133

AUDIO.META_INIT_L (u8)**Symbol:** AUDIO_META_INIT_L**Summary:** SID init address low byte when applicable.**Value:** \$BB15 (\$BB15)**Source:** audio.inc line 128**AUDIO.META_LOAD_H (u8)****Symbol:** AUDIO_META_LOAD_H**Summary:** SID load address high byte when applicable.**Value:** \$BB14 (\$BB14)**Source:** audio.inc line 123**AUDIO.META_LOAD_L (u8)****Symbol:** AUDIO_META_LOAD_L**Summary:** SID load address low byte when applicable.**Value:** \$BB13 (\$BB13)**Source:** audio.inc line 118**AUDIO.META_PLAY_H (u8)****Symbol:** AUDIO_META_PLAY_H**Summary:** SID play address high byte when applicable.**Value:** \$BB18 (\$BB18)**Source:** audio.inc line 143**AUDIO.META_PLAY_L (u8)****Symbol:** AUDIO_META_PLAY_L**Summary:** SID play address low byte when applicable.**Value:** \$BB17 (\$BB17)**Source:** audio.inc line 138**AUDIO.META_SIZE_H (u8)****Symbol:** AUDIO_META_SIZE_H**Summary:** High byte of active music file size.**Value:** \$BAB2 (\$BAB2)**Source:** audio.inc line 98**AUDIO.META_SIZE_L (u8)****Symbol:** AUDIO_META_SIZE_L**Summary:** Low byte of active music file size.**Value:** \$BAB1 (\$BAB1)**Source:** audio.inc line 93

AUDIO.META_SONGS (u8)**Symbol:** AUDIO_META_SONGS**Summary:** SID subtune count or MIDI track/song count when known.**Value:** \$BB19 (\$BB19)**Source:** audio.inc line 148**AUDIO.META_SOUNDFONT** (bytes)**Symbol:** AUDIO_META_SOUNDFONT**Summary:** 64-byte null-padded active MIDI/WTS soundfont name.**Value:** \$BB8E (\$BB8E)**Source:** audio.inc line 168**AUDIO.META_TITLE** (bytes)**Symbol:** AUDIO_META_TITLE**Summary:** 32-byte null-padded active music title.**Value:** \$BAB3 (\$BAB3)**Source:** audio.inc line 103**AUDIO.META_TYPE** (u8)**Symbol:** AUDIO_META_TYPE**Summary:** Active music metadata file type: 1=SID, 3=MIDI/WTS.**Value:** \$BAB0 (\$BAB0)**Source:** audio.inc line 88**AUDIO.MIDPLAY** (routine)**Symbol:** audio_midplay**Summary:** Play the MIDI file named by FIO.NAME/FIO.NAMELEN.**ABI:** pseudo-register**Outputs:**

- A: 0 on success, nonzero on error

Requires: FIO.NAME, FIO.NAMELEN**Source:** audio.s line 529**AUDIO.MIDPLAY_FILE** (routine)**Symbol:** audio_midplay_file**Summary:** Copy a pointer-based MIDI filename into FIO.NAME and start MIDI playback.**ABI:** pseudo-register**Outputs:**

- A: 0 on success, nonzero on error

Requires: FIO_ARG_NAMELEN, FIO_ARG_NAMEPTR_L, FIO_ARG_NAMEPTR_H**Source:** audio.s line 584

AUDIO.MIDSTOP (routine)**Symbol:** `audio_midstop`**Summary:** Stop MIDI file playback.**ABI:** none**Outputs:**

- A: 0 on success, nonzero on error

Source: `audio.s` line 542**AUDIO.MUSIC_LOOP** (routine)**Symbol:** `audio_music_loop`**Summary:** Enable or disable music looping. `AUDIO.NOTE` is 0=off, nonzero=on.**ABI:** pseudo-register**Outputs:**

- A: 0 on success, nonzero on error

Requires: `AUDIO.NOTE`**Source:** `audio.s` line 672**AUDIO.MUSIC_NOTE** (routine)**Symbol:** `audio_music_note`**Summary:** Return the current MIDI note for a music voice.**ABI:** register**Inputs:**

- X: Voice number, 1..14

Outputs:

- A: Current MIDI note, or 0 if out of range or silent

Source: `audio.s` line 739**AUDIO.MUSIC_PLAY** (routine)**Symbol:** `audio_music_play`**Summary:** Start queued music playback.**ABI:** none**Outputs:**

- A: 0 on success, nonzero on error

Source: `audio.s` line 631**AUDIO.MUSIC_PLAYING** (routine)**Symbol:** `audio_music_playing`**Summary:** Return nonzero if music is playing.**ABI:** none**Outputs:**

- A: 1 if playing, 0 if idle

Source: `audio.s` line 723

AUDIO.MUSIC_PRIORITY (routine)

Symbol: `audio_music_priority`

Summary: Set hosted MML priority from `AUDIO.NOTE` and following pseudo-register bytes.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: `AUDIO.NOTE`

Source: `audio.s` line 685

AUDIO.MUSIC_SEQUENCE (routine)

Symbol: `audio_music_sequence`

Summary: Queue an MML sequence for `AUDIO.VOICE` using `AUDIO.STRL/H` and `AUDIO.STRLLEN`.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: `AUDIO.VOICE`, `AUDIO.STRL`, `AUDIO.STRH`, `AUDIO.STRLLEN`

Source: `audio.s` line 618

AUDIO.MUSIC_STOP (routine)

Symbol: `audio_music_stop`

Summary: Stop music playback.

ABI: none

Outputs:

- A: 0 on success, nonzero on error

Source: `audio.s` line 647

AUDIO.MUSIC_TEMPO (routine)

Symbol: `audio_music_tempo`

Summary: Set music tempo from `AUDIO.NOTE/AUDIO.DURATION` as a 16-bit BPM value.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: `AUDIO.NOTE`, `AUDIO.DURATION`

Source: `audio.s` line 659

AUDIO.NOTE (u8)**Symbol:** AUDIO_NOTE**Summary:** MIDI note for AUDIO.SOUND.**Value:** FIO_SRCL (\$B9A4)**Source:** audio.inc line 189**AUDIO.PLAY_SOUND** (routine)**Symbol:** audio_play_sound**Summary:** Play a one-shot note using A/X/Y register arguments.**ABI:** register**Inputs:**

- A: MIDI note
- X: Duration in 60 Hz frames
- Y: Instrument slot

Outputs:

- A: 0 on success, nonzero on error

Source: audio.s line 176**AUDIO.RELEASE** (u8)**Symbol:** AUDIO_RELEASE**Summary:** Release nibble for AUDIO.INSTRUMENT_SET.**Value:** FIO_SIZEH (\$B9A9)**Source:** audio.inc line 241**AUDIO.SET_VOLUME** (routine)**Symbol:** audio_set_volume**Summary:** Set master or per-voice volume using A/X register arguments.**ABI:** register**Inputs:**

- A: Volume level
- X: Voice selector, 0=master

Outputs:

- A: 0 on success, nonzero on error

Source: audio.s line 193**AUDIO.SFLOAD** (routine)**Symbol:** audio_sfload**Summary:** Load the soundfont named by FIO.NAME/FIO.NAMELEN.**ABI:** pseudo-register**Outputs:**

- A: 0 on success, nonzero on error

Requires: FIO.NAME, FIO.NAMELEN

Source: audio.s line 554

AUDIO.SFLOAD_FILE (routine)

Symbol: audio_sfload_file

Summary: Copy a pointer-based soundfont filename into FIO.NAME and load it.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: FIO_ARG_NAMELEN, FIO_ARG_NAMEPTR_L, FIO_ARG_NAMEPTR_H

Source: audio.s line 600

AUDIO.SFX_PLAYING (routine)

Symbol: audio_sfx_playing

Summary: Return nonzero if the hosted music engine reports a sound effect.

ABI: none

Outputs:

- A: 1 if playing, 0 if idle

Source: audio.s line 710

AUDIO.SIDPLAY (routine)

Symbol: audio_sidplay

Summary: Play the SID file named by FIO.NAME/FIO.NAMELEN. AUDIO.NOTE is reused as the 1-based song number.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: FIO.NAME, FIO.NAMELEN, AUDIO.NOTE

Source: audio.s line 500

AUDIO.SIDPLAY_FILE (routine)

Symbol: audio_sidplay_file

Summary: Copy a pointer-based SID filename into FIO.NAME and start SID playback.

ABI: pseudo-register

Outputs:

- A: 0 on success, nonzero on error

Requires: FIO_ARG_NAMELEN, FIO_ARG_NAMEPTR_L, FIO_ARG_NAMEPTR_H, AUDIO_NOTE

Source: audio.s line 568

AUDIO.SIDSTOP (routine)**Symbol:** `audio_sidstop`**Summary:** Stop SID file playback.**ABI:** none**Outputs:**

- A: 0 on success, nonzero on error

Source: `audio.s` line 517**AUDIO.SOUND** (routine)**Symbol:** `audio_sound`**Summary:** Start a fire-and-forget SID note from `AUDIO.NOTE`, `AUDIO.DURATION`, and `AUDIO.INSTRUMENT`.**ABI:** pseudo-register**Outputs:**

- A: 0 on success, nonzero on error

Requires: `AUDIO.NOTE`, `AUDIO.DURATION`, `AUDIO.INSTRUMENT`**Source:** `audio.s` line 208**AUDIO.STATUS** (routine)**Symbol:** `audio_status`**Summary:** Return raw music status bits.**ABI:** none**Outputs:**

- A: Raw `MUSIC_STATUS` byte

Source: `audio.s` line 698**AUDIO.STATUS_LOADING** (const)**Symbol:** `AUDIO_STATUS_LOADING`**Summary:** `MUSIC_STATUS` bit set while hosted SID/MIDI/soundfont assets are loading.**Value:** `MUSIC_STATUS_LOADING` (\$10)**Source:** `audio.inc` line 78**AUDIO.STATUS_MUSIC** (const)**Symbol:** `AUDIO_STATUS_MUSIC`**Summary:** `MUSIC_STATUS` bit set while hosted music playback is active.**Value:** `MUSIC_STATUS_MUSIC` (\$02)**Source:** `audio.inc` line 63**AUDIO.STATUS_SFX** (const)**Symbol:** `AUDIO_STATUS_SFX`**Summary:** `MUSIC_STATUS` bit set while a hosted sound effect is active.**Value:** `MUSIC_STATUS_SFX` (\$01)

Source: audio.inc line 58

AUDIO.STATUS_SID (const)

Symbol: AUDIO_STATUS_SID

Summary: MUSIC_STATUS bit set when the current music source uses SID voices.

Value: MUSIC_STATUS_SID (\$04)

Source: audio.inc line 68

AUDIO.STATUS_WTS (const)

Symbol: AUDIO_STATUS_WTS

Summary: MUSIC_STATUS bit set when the current music source uses WTS/MIDI voices.

Value: MUSIC_STATUS_WTS (\$08)

Source: audio.inc line 73

AUDIO.STRH (u8)

Symbol: AUDIO_STRH

Summary: High byte of MML string pointer for AUDIO.MUSIC_SEQUENCE.

Value: FIO_ENDH (\$B9A7)

Source: audio.inc line 252

AUDIO.STRL (u8)

Symbol: AUDIO_STRL

Summary: Low byte of MML string pointer for AUDIO.MUSIC_SEQUENCE.

Value: FIO_ENDL (\$B9A6)

Source: audio.inc line 247

AUDIO.STRLEN (u8)

Symbol: AUDIO_STRLEN

Summary: MML string length for AUDIO.MUSIC_SEQUENCE.

Value: FIO_NAMELEN (\$B9A3)

Source: audio.inc line 257

AUDIO.SUSTAIN (u8)

Symbol: AUDIO_SUSTAIN

Summary: Sustain nibble for AUDIO.INSTRUMENT_SET.

Value: FIO_SIZELEN (\$B9A8)

Source: audio.inc line 236

AUDIO.TICK (routine)

Symbol: audio_tick

Summary: Advance fire-and-forget SID sound-effect durations by one frame.

ABI: none

Source: audio.s line 393

AUDIO.VOICE (u8)**Symbol:** AUDIO_VOICE**Summary:** Voice selector. Use 0 for SID master volume or 1..6 for SID voice volume.**Value:** FIO_SRCH (\$B9A5)**Source:** audio.inc line 210**AUDIO.VOLUME** (routine)**Symbol:** audio_volume**Summary:** Set SID master or per-voice volume from AUDIO.VOLUME_LEVEL and AUDIO.VOICE.**ABI:** pseudo-register**Outputs:**

- A: 0 on success, nonzero on error

Requires: AUDIO.VOLUME_LEVEL, AUDIO.VOICE**Source:** audio.s line 271**AUDIO.VOLUME_LEVEL** (u8)**Symbol:** AUDIO_VOLUME**Summary:** Volume level for AUDIO.VOLUME.**Value:** FIO_SRCL (\$B9A4)**Source:** audio.inc line 205**AUDIO.WAVEFORM** (u8)**Symbol:** AUDIO_WAVEFORM**Summary:** SID waveform/control byte for AUDIO.INSTRUMENT_SET.**Value:** FIO_SRCH (\$B9A5)**Source:** audio.inc line 221

4.4 Video and Graphics Controller

Generated from vgc.s.

VGC.CIRCLE (routine)**Symbol:** vgc_circle**Summary:** Issue the VGC circle/ellipse command using VGC.P0..VGC.P7.**Requires:** VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5, VGC_P6, VGC_P7**Source:** vgc.s line 350**VGC.CLS** (routine)**Symbol:** vgc_cls**Summary:** Clear the text screen through the VGC character output register.**Source:** vgc.s line 99

VGC.COMMAND (routine)

Symbol: `vgc_cmd`

Summary: Issue a raw VGC command without waiting for completion.

Inputs:

- A: VGC command byte.

Source: `vgc.s` line 53

VGC.DISPLAY_OFF (routine)

Symbol: `vgc_display_off`

Summary: Blank VGC output while keeping video timing active.

Source: `vgc.s` line 144

VGC.DISPLAY_ON (routine)

Symbol: `vgc_display_on`

Summary: Restore full VGC output brightness.

Source: `vgc.s` line 135

VGC.EXEC (routine)

Symbol: `vgc_exec`

Summary: Issue a raw VGC command and wait for completion.

Inputs:

- A: VGC command byte.

Source: `vgc.s` line 76

VGC.FILL (routine)

Symbol: `vgc_fill`

Summary: Issue the VGC filled rectangle command using VGC.P0..VGC.P7.

Requires: VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5, VGC_P6, VGC_P7

Source: `vgc.s` line 377

VGC.FLASH_OFF (routine)

Symbol: `vgc_flash_off`

Summary: Disable flashing text.

Source: `vgc.s` line 295

VGC.FLASH_ON (routine)

Symbol: `vgc_flash_on`

Summary: Enable flashing text.

Source: `vgc.s` line 285

VGC.GCLS (routine)**Symbol:** `vgc_gcls`**Summary:** Issue the VGC graphics clear command.**Source:** `vgc.s` line 306**VGC.GCOLOR** (routine)**Symbol:** `vgc_gcolor`**Summary:** Issue the VGC graphics colour command using VGC.P0.**Requires:** VGC_P0**Source:** `vgc.s` line 314**VGC.GTEXT** (routine)**Symbol:** `vgc_gtext`**Summary:** Issue the VGC graphics text command using VGC.P0..VGC.P5 and FIO.NAME.**Requires:** VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5, FIO_NAME, FIO_NAMELEN**Source:** `vgc.s` line 359**VGC.IRQ_ACK** (routine)**Symbol:** `vgc_irq_ack`**Summary:** Acknowledge pending VGC IRQ source bits.**Inputs:**

- A: VGC IRQ source mask.

Source: `vgc.s` line 189**VGC.IRQ_DISABLE** (routine)**Symbol:** `vgc_irq_disable`**Summary:** Disable one or more VGC IRQ source bits.**Inputs:**

- A: VGC IRQ source mask.

Source: `vgc.s` line 178**VGC.IRQ_ENABLE** (routine)**Symbol:** `vgc_irq_enable`**Summary:** Enable one or more VGC IRQ source bits and enable maskable CPU IRQs.**Inputs:**

- A: VGC IRQ source mask.

Source: `vgc.s` line 167

VGC.IRQ_INSTALL (routine)**Symbol:** `vgc_irq_install`**Summary:** Install the CPU IRQ vector used when enabled VGC IRQ sources fire.**Inputs:**

- A: Handler address high byte.
- Y: Handler address low byte.

Source: `vgc.s` line 153**VGC.LINE** (routine)**Symbol:** `vgc_line`**Summary:** Issue the VGC line command using VGC.P0..VGC.P7.**Requires:** VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5, VGC_P6, VGC_P7**Source:** `vgc.s` line 341**VGC.LOCATE** (routine)**Symbol:** `vgc_locate`**Summary:** Set the text cursor position from VGC.P0/VGC.P1.**Inputs:**

- VGC_P0: Cursor X column.
- VGC_P1: Cursor Y row.

Source: `vgc.s` line 198**VGC.MEM_READ** (routine)**Symbol:** `vgc_mem_read`**Summary:** Read one byte from VGC memory using VGC.P0..VGC.P4.**Outputs:**

- VGC_P3: Read byte.

Requires: VGC_P0, VGC_P1, VGC_P2, VGC_P4**Source:** `vgc.s` line 229**VGC.MEM_WRITE** (routine)**Symbol:** `vgc_mem_write`**Summary:** Write one byte to VGC memory using VGC.P0..VGC.P4.**Requires:** VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4**Source:** `vgc.s` line 239**VGC.PAINT** (routine)**Symbol:** `vgc_paint`**Summary:** Issue the VGC paint command using VGC.P0..VGC.P3.**Requires:** VGC_P0, VGC_P1, VGC_P2, VGC_P3

Source: `vgc.s` line 386

VGC.PLOT (routine)

Symbol: `vgc_plot`

Summary: Issue the VGC plot command using VGC.P0..VGC.P3.

Requires: VGC_P0, VGC_P1, VGC_P2, VGC_P3

Source: `vgc.s` line 323

VGC.RECT (routine)

Symbol: `vgc_rect`

Summary: Issue the VGC rectangle command using VGC.P0..VGC.P7.

Requires: VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5, VGC_P6, VGC_P7

Source: `vgc.s` line 368

VGC.REVERSE (routine)

Symbol: `vgc_reverse_default`

Summary: Enable reverse text using the current foreground/background colours.

Source: `vgc.s` line 250

VGC.REVERSE_EXPLICIT (routine)

Symbol: `vgc_reverse_explicit`

Summary: Enable reverse text with explicit packed foreground/background colours.

Inputs:

- A: Packed reverse attribute, low nibble foreground and high nibble background.

Source: `vgc.s` line 262

VGC.REVERSE_OFF (routine)

Symbol: `vgc_reverse_off`

Summary: Disable reverse text.

Source: `vgc.s` line 275

VGC.SET_BG (routine)

Symbol: `vgc_set_bg`

Summary: Set the current text background colour.

Inputs:

- X: Background colour byte.

Source: `vgc.s` line 117

VGC.SET_BORDER (routine)

Symbol: `vgc_set_border`

Summary: Set the VGC border colour.

Inputs:

- X: Border colour byte.

Source: `vgc.s` line 126

VGC.SET_FG (routine)

Symbol: `vgc_set_fg`

Summary: Set the current text foreground colour.

Inputs:

- X: Foreground colour byte.

Source: `vgc.s` line 108

VGC.SET_FONT (routine)

Symbol: `vgc_set_font`

Summary: Select the active text font slot.

Inputs:

- X: Font slot, 0..7.

Source: `vgc.s` line 220

VGC.SET_MODE (routine)

Symbol: `vgc_set_mode`

Summary: Set the active VGC graphics/text mode.

Inputs:

- X: Mode byte.

Source: `vgc.s` line 211

VGC.UNPLOT (routine)

Symbol: `vgc_unplot`

Summary: Issue the VGC unplot command using `VGC.P0..VGC.P3`.

Requires: `VGC_P0`, `VGC_P1`, `VGC_P2`, `VGC_P3`

Source: `vgc.s` line 332

VGC.VSYNC (routine)

Symbol: `vgc_vsync`

Summary: Wait until the VGC frame counter advances.

Outputs:

- A: Last observed frame counter value.

Source: `vgc.s` line 87

VGC.WAIT_COMMAND (routine)**Symbol:** `vgc_wait_cmd`**Summary:** Wait for the active VGC command to complete.**Source:** `vgc.s` line 63

4.5 Sprites

*Generated from `sprite.s`.***SPRITE.BACKGROUND_CLEAR** (routine)**Symbol:** `sprite_background_clear`**Summary:** Clear the sprite-background collision mask and acknowledge its IRQ source.**Outputs:**

- A: `VGC_IRQ_SPRBG`.

Source: `sprite.s` line 315**SPRITE.BACKGROUND_IRQ_DISABLE** (routine)**Symbol:** `sprite_background_irq_disable`**Summary:** Disable the VGC sprite-background IRQ source.**Source:** `sprite.s` line 352**SPRITE.BACKGROUND_IRQ_ENABLE** (routine)**Symbol:** `sprite_background_irq_enable`**Summary:** Clear stale sprite-background collisions, enable the VGC sprite-background IRQ source, and enable CPU IRQs.**Source:** `sprite.s` line 340**SPRITE.BACKGROUND_MASK** (routine)**Symbol:** `sprite_background_collision_mask`**Summary:** Read the 16-bit sprite-background collision status mask without clearing it.**Outputs:**

- A: Background collision status high byte for sprites 8-15.
- Y: Background collision status low byte for sprites 0-7.

Source: `sprite.s` line 304**SPRITE.BACKGROUND_READ_CLEAR** (routine)**Symbol:** `sprite_background_read_clear`**Summary:** Read the 16-bit sprite-background collision mask, clear it, and acknowledge its IRQ source.**Outputs:**

- A: Background collision status high byte for sprites 8-15.

- Y: Background collision status low byte for sprites 0-7.

Source: `sprite.s` line 327

SPRITE.BACKGROUND_STATUS (routine)

Symbol: `sprite_background_collision_status`

Summary: Read the low byte of the sprite-background collision status register.

Outputs:

- A: Background collision status byte for sprites 0-7.

Source: `sprite.s` line 295

SPRITE.CLEAR (routine)

Symbol: `sprite_clear`

Summary: Clear the shape data for the sprite in VGC.P0.

Requires: VGC_P0

Source: `sprite.s` line 91

SPRITE.COLLISION_CLEAR (routine)

Symbol: `sprite_collision_clear`

Summary: Clear the sprite-sprite collision mask and acknowledge its IRQ source.

Outputs:

- A: VGC_IRQ_SPRCOLL.

Source: `sprite.s` line 247

SPRITE.COLLISION_IRQ_DISABLE (routine)

Symbol: `sprite_collision_irq_disable`

Summary: Disable the VGC sprite collision IRQ source.

Source: `sprite.s` line 284

SPRITE.COLLISION_IRQ_ENABLE (routine)

Symbol: `sprite_collision_irq_enable`

Summary: Clear stale sprite-sprite collisions, enable the VGC sprite collision IRQ source, and enable CPU IRQs.

Source: `sprite.s` line 272

SPRITE.COLLISION_MASK (routine)

Symbol: `sprite_collision_mask`

Summary: Read the 16-bit sprite-sprite collision status mask without clearing it.

Outputs:

- A: Collision status high byte for sprites 8-15.
- Y: Collision status low byte for sprites 0-7.

Source: `sprite.s` line 236

SPRITE.COLLISION_READ_CLEAR (routine)

Symbol: `sprite_collision_read_clear`

Summary: Read the 16-bit sprite-sprite collision mask, clear it, and acknowledge its IRQ source.

Outputs:

- A: Collision status high byte for sprites 8-15.
- Y: Collision status low byte for sprites 0-7.

Source: `sprite.s` line 259

SPRITE.COLLISION_STATUS (routine)

Symbol: `sprite_collision_status`

Summary: Read the low byte of the sprite-sprite collision status register.

Outputs:

- A: Collision status byte for sprites 0-7.

Source: `sprite.s` line 227

SPRITE.COMMAND (routine)

Symbol: `sprite_command`

Summary: Issue a raw sprite VGC command without waiting for completion.

Inputs:

- A: VGC sprite command byte.

Source: `sprite.s` line 44

SPRITE.COPY (routine)

Symbol: `sprite_copy`

Summary: Copy sprite shape data from VGC.P0 to VGC.P1.

Requires: VGC_P0, VGC_P1

Source: `sprite.s` line 100

SPRITE.DISABLE (routine)

Symbol: `sprite_disable`

Summary: Disable the sprite in VGC.P0.

Requires: VGC_P0

Source: `sprite.s` line 72

SPRITE.ENABLE (routine)

Symbol: `sprite_enable`

Summary: Enable the sprite in VGC.P0.

Requires: VGC_P0
Source: `sprite.s` line 63

SPRITE.FLIP (routine)

Symbol: `sprite_flip`
Summary: Set sprite flip flags using VGC.P0=sprite and VGC.P1=flags.
Requires: VGC_P0, VGC_P1
Source: `sprite.s` line 109

SPRITE.GET_X (routine)

Symbol: `sprite_get_x`
Summary: Read a sprite's 16-bit X coordinate.
Inputs:

- X: Sprite index.

Outputs:

- A: X high byte.
- Y: X low byte.

Source: `sprite.s` line 196

SPRITE.GET_Y (routine)

Symbol: `sprite_get_y`
Summary: Read a sprite's Y coordinate.
Inputs:

- X: Sprite index.

Outputs:

- Y: Y coordinate byte.

Source: `sprite.s` line 214

SPRITE.POS (routine)

Symbol: `sprite_pos`
Summary: Move a sprite using VGC.P0=sprite, VGC.P1/P2=x, VGC.P3=y.
Requires: VGC_P0, VGC_P1, VGC_P2, VGC_P3
Source: `sprite.s` line 54

SPRITE.PRIORITY (routine)

Symbol: `sprite_priority`
Summary: Set sprite priority using VGC.P0=sprite and VGC.P1=priority.
Requires: VGC_P0, VGC_P1
Source: `sprite.s` line 118

SPRITE.REG_OFFSET (routine)**Symbol:** `sprite_reg_offset`**Summary:** Compute the VGC sprite attribute block offset for a sprite field.**Inputs:**

- X: Sprite index.
- Y: Sprite field offset.

Outputs:

- Y: VGC_SPR_BASE-relative byte offset.

Source: `sprite.s` line 131**SPRITE.ROW** (routine)**Symbol:** `sprite_row`**Summary:** Write one sprite bitmap row using VGC.P0..VGC.P9.**Requires:** VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5, VGC_P6, VGC_P7, VGC_P8, VGC_P9**Source:** `sprite.s` line 81**SPRITE.SET_REG16** (routine)**Symbol:** `sprite_set_reg16`**Summary:** Write one 16-bit sprite attribute field.**Inputs:**

- X: Sprite index.
- Y: Sprite field offset.
- NVROL: Low byte.
- NVROH: High byte.

Source: `sprite.s` line 178**SPRITE.SET_REG8** (routine)**Symbol:** `sprite_set_reg8`**Summary:** Write one 8-bit sprite attribute field.**Inputs:**

- X: Sprite index.
- Y: Sprite field offset.
- A: Field value.

Source: `sprite.s` line 162

SPRITE.SET_SHAPE (routine)**Symbol:** `sprite_set_shape`**Summary:** Set a sprite's shape index.**Inputs:**

- X: Sprite index.
- A: Shape index.

Source: `sprite.s` line 150

4.6 Meta-Sprites

*Generated from `msprite.inc`, `msprite.s`.***MSPRITE.ANIM_H** (u8)**Symbol:** `MSPRITE_ANIM_H`**Summary:** High byte of the animation descriptor pointer used by `MSPRITE.SET_ANIM`.**Source:** `msprite.inc` line 58**MSPRITE.ANIM_L** (u8)**Symbol:** `MSPRITE_ANIM_L`**Summary:** Low byte of the animation descriptor pointer used by `MSPRITE.SET_ANIM`.**Source:** `msprite.inc` line 53**MSPRITE.COMMIT** (routine)**Symbol:** `msprite_commit`**Summary:** Write all dirty visible meta-sprite objects to VGC hardware sprite registers.**Outputs:**

- A: 0 on success.

Source: `msprite.s` line 407**MSPRITE.COMMIT_ONE** (routine)**Symbol:** `msprite_commit_one`**Summary:** Write one visible meta-sprite object to VGC hardware sprite registers even if it is not dirty.**Inputs:**

- A: Object handle.

Outputs:

- A: Result code.

Source: `msprite.s` line 428

MSPRITE.DESC_H (u8)**Symbol:** MSPRITE_DESC_H**Summary:** High byte of the visual descriptor pointer used by MSPRITE.SPAWN.**Source:** msprite.inc line 48**MSPRITE.DESC_L** (u8)**Symbol:** MSPRITE_DESC_L**Summary:** Low byte of the visual descriptor pointer used by MSPRITE.SPAWN.**Source:** msprite.inc line 43**MSPRITE.DESTROY** (routine)**Symbol:** msprite_destroy**Summary:** Destroy one meta-sprite object, disable its hardware sprites, and release its sprite allocation.**Inputs:**

- A: Object handle.

Outputs:

- A: Result code.

Source: msprite.s line 188**MSPRITE.HIDE** (routine)**Symbol:** msprite_hide**Summary:** Mark one object hidden and immediately disable its child hardware sprites.**Inputs:**

- A: Object handle.

Outputs:

- A: Result code.

Source: msprite.s line 222**MSPRITE.INIT** (routine)**Symbol:** msprite_init**Summary:** Clear the meta-sprite object table, release all hardware sprites, and disable every hardware sprite.**Outputs:**

- A: 0 on success.

Source: msprite.s line 75

MSPRITE.RESULT (u8)**Symbol:** MSPRITE_RESULT**Summary:** Result code from the last meta-sprite operation.**Source:** msprite.inc line 63**MSPRITE.SET_ANIM** (routine)**Symbol:** msprite_set_anim**Summary:** Attach an optional linear animation descriptor to an object and reset its frame timer.**Inputs:**

- A: Object handle.

Outputs:

- A: Result code.

Requires: MSPRITE_ANIM_L, MSPRITE_ANIM_H**Source:** msprite.s line 285**MSPRITE.SET_FRAME** (routine)**Symbol:** msprite_set_frame**Summary:** Set an object's animation frame and mark it dirty.**Inputs:**

- A: Object handle.
- X: Frame index.

Outputs:

- A: Result code.

Source: msprite.s line 266**MSPRITE.SET_POS** (routine)**Symbol:** msprite_set_pos**Summary:** Set an object's position and mark it dirty.**Inputs:**

- A: Object handle.
- NVR0L: X low byte.
- NVR0H: X high byte.
- Y: Y coordinate byte.

Outputs:

- A: Result code.

Source: msprite.s line 242

MSPRITE.SET_PRIORITY (routine)**Symbol:** `msprite_set_priority`**Summary:** Set the shared priority used for all parts of an object.**Inputs:**

- A: Object handle.
- X: Sprite priority, 0 behind, 1 between, 2 front.

Outputs:

- A: Result code.

Source: `msprite.s` line 306**MSPRITE.SET_TRANSCOLOR** (routine)**Symbol:** `msprite_set_transcolor`**Summary:** Set the transparent color used for all parts of an object.**Inputs:**

- A: Object handle.
- X: Transparent color index.

Outputs:

- A: Result code.

Source: `msprite.s` line 329**MSPRITE.SHOW** (routine)**Symbol:** `msprite_show`**Summary:** Mark one object visible and dirty so its child hardware sprites are enabled on the next commit.**Inputs:**

- A: Object handle.

Outputs:

- A: Result code.

Source: `msprite.s` line 206**MSPRITE.SPAWN** (routine)**Symbol:** `msprite_spawn`**Summary:** Allocate hardware sprites for `MSPRITE.DESC` and return a new object handle.**Outputs:**

- A: Object handle on success, `$FF` on failure.
- `MSPRITE_RESULT`: Result code.

Requires: MSPRITE_DESC_L, MSPRITE_DESC_H

Source: msprite.s line 117

MSPRITE.TICK (routine)

Symbol: msprite_tick

Summary: Advance animation timers for every active object with an attached animation descriptor.

Outputs:

- A: 0 on success.

Source: msprite.s line 347

4.7 Vector Sprites

Generated from vsprite.inc, vsprite.s.

VSPRITE.BGADDRH (u8)

Symbol: VSPRITE_BGADDRH

Summary: High byte of the caller-owned saved-background buffer address. Use 0 for CPU RAM.

Source: vsprite.inc line 199

VSPRITE.BGADDRL (u8)

Symbol: VSPRITE_BGADDRL

Summary: Low byte of the caller-owned saved-background buffer address.

Source: vsprite.inc line 189

VSPRITE.BGADDRM (u8)

Symbol: VSPRITE_BGADDRM

Summary: Middle byte of the caller-owned saved-background buffer address.

Source: vsprite.inc line 194

VSPRITE.BGSPACE (u8)

Symbol: VSPRITE_BGSPACE

Summary: Blitter memory space for the saved-background buffer. Use BLT_SPACE_CPU for normal RAM buffers.

Source: vsprite.inc line 184

VSPRITE.BGSTRH (u8)

Symbol: VSPRITE_BGSTRH

Summary: High byte of the saved-background buffer row stride.

Source: vsprite.inc line 209

VSPRITE.BGSTRL (u8)**Symbol:** VSPRITE_BGSTRL**Summary:** Low byte of the saved-background buffer row stride.**Source:** vsprite.inc line 204**VSPRITE.BLIT** (routine)**Symbol:** vsprite_blit**Summary:** Copy the configured virtual sprite rectangle with the blitter.**Inputs:**

- VSPRITE_FLAGS: Set bit 0 to enable color-key transparency.
- VSPRITE_COLORKEY: Source byte skipped when color-key mode is enabled.
- VGC_GFXTRANS: Graphics-plane color treated as transparent by display composition; set this to a non-black color when blitting visible palette-0 black pixels.

Outputs:

- A: 0 on success, 1 on error.

Requires: VSPRITE_SRCSPACE, VSPRITE_DSTSPACE, VSPRITE_SRCADDR*,
VSPRITE_DSTADDR*, VSPRITE_WIDTH*, VSPRITE_HEIGHT*, VSPRITE_SRCSTR*,
VSPRITE_DSTSTR*

Source: vsprite.s line 87**VSPRITE.COLORKEY** (u8)**Symbol:** VSPRITE_COLORKEY**Summary:** Source byte skipped by color-keyed blits. For graphics-plane virtual sprites, normally match this to VGC.GFXTRANS.**Source:** vsprite.inc line 214**VSPRITE.DESC_FLAG_COLORKEY** (const)**Symbol:** VSPRITE_DESC_FLAG_COLORKEY**Summary:** Descriptor flag enabling source color-key transparency.**Value:** VSPRITE_FLAG_COLORKEY (\$01)**Source:** vsprite.inc line 55**VSPRITE.DESC_SAVED** (const)**Symbol:** VSPRITE_DESC_SAVED**Summary:** Internal descriptor flag set when old X/Y has a saved background to restore.**Value:** \$40 (\$40)**Source:** vsprite.inc line 61**VSPRITE.DESC_VISIBLE** (const)**Symbol:** VSPRITE_DESC_VISIBLE**Summary:** Descriptor flag enabling scene save/draw for this virtual sprite.

Value: \$80 (\$80)
Source: vsprite.inc line 67

VSPRITE.DSTADDRH (u8)

Symbol: VSPRITE_DSTADDRH
Summary: High byte of the virtual-sprite destination address. VSPRITE.GFX_* routines compute this from VSPRITE.X/Y.
Source: vsprite.inc line 139

VSPRITE.DSTADDRL (u8)

Symbol: VSPRITE_DSTADDRL
Summary: Low byte of the virtual-sprite destination address. VSPRITE.GFX_* routines compute this from VSPRITE.X/Y.
Source: vsprite.inc line 129

VSPRITE.DSTADDRM (u8)

Symbol: VSPRITE_DSTADDRM
Summary: Middle byte of the virtual-sprite destination address. VSPRITE.GFX_* routines compute this from VSPRITE.X/Y.
Source: vsprite.inc line 134

VSPRITE.DSTSPACE (u8)

Symbol: VSPRITE_DSTSPACE
Summary: Blitter memory space for the virtual-sprite destination rectangle.
Source: vsprite.inc line 109

VSPRITE.DSTSTRH (u8)

Symbol: VSPRITE_DSTSTRH
Summary: High byte of the destination row stride.
Source: vsprite.inc line 179

VSPRITE.DSTSTRL (u8)

Symbol: VSPRITE_DSTSTRL
Summary: Low byte of the destination row stride.
Source: vsprite.inc line 174

VSPRITE.FILL (routine)

Symbol: vsprite_fill
Summary: Fill the configured destination rectangle with VSPRITE.FILLVALUE.
Outputs:

- A: 0 on success, 1 on error.

Requires: VSPRITE_DSTSPACE, VSPRITE_DSTADDR*, VSPRITE_WIDTH*, VSPRITE_HEIGHT*, VSPRITE_DSTSTR*

Source: `vsprite.s` line 144

VSPRITE.FILLVALUE (u8)

Symbol: `VSPRITE_FILLVALUE`

Summary: Byte value used by `VSPRITE.FILL` and `VSPRITE.GFX_FILL`.

Source: `vsprite.inc` line 219

VSPRITE.FLAGS (u8)

Symbol: `VSPRITE_FLAGS`

Summary: Bitfield controlling virtual-sprite blits. Set `VSPRITE.FLAG_COLORKEY` to enable source transparency.

Source: `vsprite.inc` line 224

VSPRITE.FLAG_COLORKEY (const)

Symbol: `VSPRITE_FLAG_COLORKEY`

Summary: Enable source color-key transparency for `VSPRITE.BLIT` and `VSPRITE.GFX_BLIT`.

Value: `$01` (`$01`)

Source: `vsprite.inc` line 49

VSPRITE.GFX_ADDR (routine)

Symbol: `vsprite_gfx_addr`

Summary: Convert `VSPRITE.X/Y` to a linear VGC graphics-plane destination address.

Outputs:

- `VSPRITE_DSTADDR*`: $y * 320 + x$.

Requires: `VSPRITE_XL`, `VSPRITE_XH`, `VSPRITE_Y`

Source: `vsprite.s` line 175

VSPRITE.GFX_BLIT (routine)

Symbol: `vsprite_gfx_blit`

Summary: Copy the configured virtual sprite to the VGC graphics plane at `X/Y`.

Inputs:

- `VSPRITE_COLORKEY`: Source transparent color when `VSPRITE_FLAG_COLORKEY` is set; normally match this to `VGC_GFXTRANS` for graphics-plane virtual sprites.

Outputs:

- `A`: 0 on success, 1 on error.

Requires: `VSPRITE_XL`, `VSPRITE_XH`, `VSPRITE_Y`, `VSPRITE_SRCSPACE`, `VSPRITE_SRCADDR*`, `VSPRITE_WIDTH*`, `VSPRITE_HEIGHT*`, `VSPRITE_SRCSTR*`

Source: `vsprite.s` line 220

VSPRITE.GFX_FILL (routine)**Symbol:** `vsprite_gfx_fill`**Summary:** Fill a graphics-plane rectangle at VSPRITE.X/Y.**Outputs:**

- A: 0 on success, 1 on error.

Requires: VSPRITE_XL, VSPRITE_XH, VSPRITE_Y, VSPRITE_WIDTH*, VSPRITE_HEIGHT*, VSPRITE_FILLVALUE**Source:** `vsprite.s` line 237**VSPRITE.GFX_RESTORE_BG** (routine)**Symbol:** `vsprite_gfx_restore_bg`**Summary:** Restore a caller-owned VSPRITE.BG* buffer into the graphics plane at VSPRITE.X/Y.**Outputs:**

- A: 0 on success, 1 on error.

Requires: VSPRITE_XL, VSPRITE_XH, VSPRITE_Y, VSPRITE_WIDTH*, VSPRITE_HEIGHT*, VSPRITE_BGSPACE, VSPRITE_BGADDR*, VSPRITE_BGSTR***Source:** `vsprite.s` line 295**VSPRITE.GFX_SAVE_BG** (routine)**Symbol:** `vsprite_gfx_save_bg`**Summary:** Save a graphics-plane rectangle under VSPRITE.X/Y into the caller-owned VSPRITE.BG* buffer.**Outputs:**

- A: 0 on success, 1 on error.

Requires: VSPRITE_XL, VSPRITE_XH, VSPRITE_Y, VSPRITE_WIDTH*, VSPRITE_HEIGHT*, VSPRITE_BGSPACE, VSPRITE_BGADDR*, VSPRITE_BGSTR***Source:** `vsprite.s` line 253**VSPRITE.HEIGHTH** (u8)**Symbol:** VSPRITE_HEIGHTH**Summary:** High byte of the rectangle height in rows.**Source:** `vsprite.inc` line 159**VSPRITE.HEIGHTL** (u8)**Symbol:** VSPRITE_HEIGHTL**Summary:** Low byte of the rectangle height in rows.**Source:** `vsprite.inc` line 154

VSPRITE.SCENE_ADDRH (u8)**Symbol:** VSPRITE_SCENE_ADDRH**Summary:** High byte of the caller-owned scene descriptor table.**Source:** vsprite.inc line 250**VSPRITE.SCENE_ADDRL** (u8)**Symbol:** VSPRITE_SCENE_ADDRL**Summary:** Low byte of the caller-owned scene descriptor table.**Source:** vsprite.inc line 245**VSPRITE.SCENE_BEGIN** (routine)**Symbol:** vsprite_scene_begin**Summary:** Restore all saved old backgrounds in the configured scene descriptor list.**Requires:** VSPRITE_SCENE_ADDRL, VSPRITE_SCENE_ADDRH, VSPRITE_SCENE_COUNT**Source:** vsprite.s line 337**VSPRITE.SCENE_COMMIT** (routine)**Symbol:** vsprite_scene_commit**Summary:** Save all current backgrounds, then draw all visible descriptors in list order.**Requires:** VSPRITE_SCENE_ADDRL, VSPRITE_SCENE_ADDRH, VSPRITE_SCENE_COUNT**Source:** vsprite.s line 364**VSPRITE.SCENE_COMMIT_ATOMIC** (routine)**Symbol:** vsprite_scene_commit_atomic**Summary:** Compose a scene into a caller-owned work buffer, then commit the completed dirty rectangle to graphics memory.**Requires:** VSPRITE_SCENE_ADDRL, VSPRITE_SCENE_ADDRH, VSPRITE_SCENE_COUNT, VSPRITE_SCENE_X*, VSPRITE_SCENE_Y, VSPRITE_SCENE_WIDTH*, VSPRITE_SCENE_HEIGHT*, VSPRITE_SCENE_WORK***Source:** vsprite.s line 396**VSPRITE.SCENE_COUNT** (u8)**Symbol:** VSPRITE_SCENE_COUNT**Summary:** Number of descriptors in the scene table.**Source:** vsprite.inc line 255**VSPRITE.SCENE_DRAW** (routine)**Symbol:** vsprite_scene_draw**Summary:** Draw all visible descriptors in list order without saving backgrounds.**Requires:** VSPRITE_SCENE_ADDRL, VSPRITE_SCENE_ADDRH, VSPRITE_SCENE_COUNT**Source:** vsprite.s line 374

VSPRITE.SCENE_HEIGHTH (u8)

Symbol: VSPRITE_SCENE_HEIGHTH

Summary: High byte of the dirty rectangle height for atomic scene commits.

Source: vsprite.inc line 291

VSPRITE.SCENE_HEIGHTL (u8)

Symbol: VSPRITE_SCENE_HEIGHTL

Summary: Low byte of the dirty rectangle height for atomic scene commits.

Source: vsprite.inc line 286

VSPRITE.SCENE_WIDTHH (u8)

Symbol: VSPRITE_SCENE_WIDTHH

Summary: High byte of the dirty rectangle width for atomic scene commits.

Source: vsprite.inc line 281

VSPRITE.SCENE_WIDTHL (u8)

Symbol: VSPRITE_SCENE_WIDTHL

Summary: Low byte of the dirty rectangle width for atomic scene commits.

Source: vsprite.inc line 276

VSPRITE.SCENE_WORKADDRH (u8)

Symbol: VSPRITE_SCENE_WORKADDRH

Summary: High byte of the atomic scene work buffer address.

Source: vsprite.inc line 311

VSPRITE.SCENE_WORKADDRL (u8)

Symbol: VSPRITE_SCENE_WORKADDRL

Summary: Low byte of the atomic scene work buffer address.

Source: vsprite.inc line 301

VSPRITE.SCENE_WORKADDRM (u8)

Symbol: VSPRITE_SCENE_WORKADDRM

Summary: Middle byte of the atomic scene work buffer address.

Source: vsprite.inc line 306

VSPRITE.SCENE_WORKSPACE (u8)

Symbol: VSPRITE_SCENE_WORKSPACE

Summary: Blitter memory space for the atomic scene work buffer.

Source: vsprite.inc line 296

VSPRITE.SCENE_WORKSTRH (u8)

Symbol: VSPRITE_SCENE_WORKSTRH

Summary: High byte of the atomic scene work buffer row stride.

Source: vsprite.inc line 321

VSPRITE.SCENE_WORKSTRL (u8)

Symbol: VSPRITE_SCENE_WORKSTRL

Summary: Low byte of the atomic scene work buffer row stride.

Source: vsprite.inc line 316

VSPRITE.SCENE_XH (u8)

Symbol: VSPRITE_SCENE_XH

Summary: High byte of the graphics-plane dirty rectangle X coordinate for atomic scene commits.

Source: vsprite.inc line 266

VSPRITE.SCENE_XL (u8)

Symbol: VSPRITE_SCENE_XL

Summary: Low byte of the graphics-plane dirty rectangle X coordinate for atomic scene commits.

Source: vsprite.inc line 261

VSPRITE.SCENE_Y (u8)

Symbol: VSPRITE_SCENE_Y

Summary: Graphics-plane dirty rectangle Y coordinate for atomic scene commits.

Source: vsprite.inc line 271

VSPRITE.SRCADDRH (u8)

Symbol: VSPRITE_SRCADDRH

Summary: High byte of the virtual-sprite source address.

Source: vsprite.inc line 124

VSPRITE.SRCADDRL (u8)

Symbol: VSPRITE_SRCADDRL

Summary: Low byte of the virtual-sprite source address.

Source: vsprite.inc line 114

VSPRITE.SRCADDRM (u8)

Symbol: VSPRITE_SRCADDRM

Summary: Middle byte of the virtual-sprite source address.

Source: vsprite.inc line 119

VSPRITE.SRCSPACE (u8)**Symbol:** VSPRITE_SRCSPACE**Summary:** Blitter memory space for the virtual-sprite source rectangle.**Source:** vsprite.inc line 104**VSPRITE.SRCSTRH** (u8)**Symbol:** VSPRITE_SRCSTRH**Summary:** High byte of the source row stride.**Source:** vsprite.inc line 169**VSPRITE.SRCSTRL** (u8)**Symbol:** VSPRITE_SRCSTRL**Summary:** Low byte of the source row stride.**Source:** vsprite.inc line 164**VSPRITE.WIDTHH** (u8)**Symbol:** VSPRITE_WIDTHH**Summary:** High byte of the rectangle width in pixels/bytes.**Source:** vsprite.inc line 149**VSPRITE.WIDTHL** (u8)**Symbol:** VSPRITE_WIDTHL**Summary:** Low byte of the rectangle width in pixels/bytes.**Source:** vsprite.inc line 144**VSPRITE.XH** (u8)**Symbol:** VSPRITE_XH**Summary:** High byte of the graphics-plane X coordinate used by VSPRITE.GFX_* helpers.**Source:** vsprite.inc line 234**VSPRITE.XL** (u8)**Symbol:** VSPRITE_XL**Summary:** Low byte of the graphics-plane X coordinate used by VSPRITE.GFX_* helpers.**Source:** vsprite.inc line 229**VSPRITE.Y** (u8)**Symbol:** VSPRITE_Y**Summary:** Graphics-plane Y coordinate used by VSPRITE.GFX_* helpers.**Source:** vsprite.inc line 239

4.8 Nova UI

Generated from *nui.s*.

NUI.DIALOG_DEFAULTS (routine)

Symbol: `nui_dialog_defaults`

Summary: Set the standard centered modal-dialog geometry.

Source: `nui.s` line 49

NUI.SHOW_DIALOG (routine)

Symbol: `nui_show_dialog`

Summary: Draw the configured modal dialog without waiting for input.

Source: `nui.s` line 335

NUI.SHOW_DIALOG_WAIT (routine)

Symbol: `nui_show_dialog_wait`

Summary: Draw the configured modal dialog and wait for one key.

Source: `nui.s` line 351

NUI.SHOW_ERROR (routine)

Symbol: `nui_show_error`

Summary: Draw a modal error dialog and wait for one key.

Source: `nui.s` line 363

NUI.WAIT_KEY (routine)

Symbol: `nui_wait_key`

Summary: Wait for one keyboard byte and return it in A.

Source: `nui.s` line 375

4.9 Copper

Generated from *copper.s*.

COPPER.ADD (routine)

Symbol: `copper_add`

Summary: Add a copper operation using VGC.P0..VGC.P5.

Requires: VGC_P0, VGC_P1, VGC_P2, VGC_P3, VGC_P4, VGC_P5

Source: `copper.s` line 86

COPPER.CLEAR (routine)

Symbol: `copper_clear`

Summary: Clear the active copper list.

Source: `copper.s` line 36

COPPER.COMMAND (routine)**Symbol:** `copper_command`**Summary:** Issue a raw copper VGC command.**Inputs:**

- A: Copper VGC command byte.

Source: `copper.s` line 26**COPPER.LIST** (routine)**Symbol:** `copper_list`**Summary:** Select a copper list using VGC.P0.**Requires:** VGC_P0**Source:** `copper.s` line 60**COPPER.LIST_END** (routine)**Symbol:** `copper_list_end`**Summary:** Finish the current copper list.**Source:** `copper.s` line 69**COPPER.OFF** (routine)**Symbol:** `copper_off`**Summary:** Disable copper execution.**Source:** `copper.s` line 52**COPPER.ON** (routine)**Symbol:** `copper_on`**Summary:** Enable copper execution.**Source:** `copper.s` line 44**COPPER.SET_REG_INDEX** (routine)**Symbol:** `copper_set_reg_index`**Summary:** Set VGC.P3/P4 to a direct VGC register index.**Inputs:**

- A: VGC register index.

Source: `copper.s` line 96**COPPER.SET_SPRITE_REG** (routine)**Symbol:** `copper_set_sprite_reg`**Summary:** Set VGC.P3/P4 to a sprite attribute register address.**Inputs:**

- X: Sprite index.
- A: Sprite field offset.

Source: `copper.s` line 109

COPPER.USE (routine)

Symbol: `copper_use`

Summary: Activate a copper list using VGC.P0.

Requires: VGC_P0

Source: `copper.s` line 77

4.10 DMA

Generated from `dma.s`.

DMA.COPY (routine)

Symbol: `dma_copy`

Summary: Start a DMA copy using the DMA controller registers.

Outputs:

- A: 0 on success, 1 on error.

Requires: DMA_SRCSPACE, DMA_DSTSPACE, DMA_SRCL, DMA_SRCM, DMA_SRCH, DMA_DSTL, DMA_DSTM, DMA_DSTH, DMA_LENL, DMA_LENH, DMA_LENH

Source: `dma.s` line 20

DMA.FILL (routine)

Symbol: `dma_fill`

Summary: Start a DMA fill using the destination and length registers.

Outputs:

- A: 0 on success, 1 on error.

Requires: DMA_DSTSPACE, DMA_DSTL, DMA_DSTM, DMA_DSTH, DMA_LENL, DMA_LENH, DMA_LENH, DMA_FILLVALUE

Source: `dma.s` line 29

DMA.START_COPY (routine)

Symbol: `dma_start_copy`

Summary: Low-level DMA copy start. Call DMA.COPY for the stable public wrapper.

Outputs:

- A: 0 on success, 1 on error.

Source: `dma.s` line 42

DMA.START_FILL (routine)

Symbol: `dma_start_fill`

Summary: Low-level DMA fill start. Call DMA.FILL for the stable public wrapper.

Outputs:

- A: 0 on success, 1 on error.

Source: dma.s line 53

DMA.WAIT (routine)

Symbol: dma_wait

Summary: Wait for the DMA controller to leave BUSY and return shared status.

Outputs:

- A: 0 on success, 1 on error.

Source: dma.s line 65

4.11 Blitter

Generated from blitter.s.

BLITTER.COPY (routine)

Symbol: blitter_copy

Summary: Start a rectangular blitter copy using the BLT controller registers.

Outputs:

- A: 0 on success, 1 on error.

Requires: BLT_SRCSPACE, BLT_DSTSPACE, BLT_SRCL, BLT_SRCM, BLT_SRCH, BLT_DSTL, BLT_DSTM, BLT_DSTH, BLT_WIDTHL, BLT_WIDTHH, BLT_HEIGHTL, BLT_HEIGHTH, BLT_SRCSTRL, BLT_SRCSTRH, BLT_DSTSTRL, BLT_DSTSTRH

Source: blitter.s line 20

BLITTER.FILL (routine)

Symbol: blitter_fill

Summary: Start a rectangular blitter fill using destination, size, stride, and fill registers.

Outputs:

- A: 0 on success, 1 on error.

Requires: BLT_DSTSPACE, BLT_DSTL, BLT_DSTM, BLT_DSTH, BLT_WIDTHL, BLT_WIDTHH, BLT_HEIGHTL, BLT_HEIGHTH, BLT_DSTSTRL, BLT_DSTSTRH, BLT_FILLVALUE

Source: blitter.s line 29

BLITTER.START_COPY (routine)

Symbol: blitter_start_copy

Summary: Low-level blitter copy start. Call BLITTER.COPY for the public wrapper.

Outputs:

- A: 0 on success, 1 on error.

Source: blitter.s line 45

BLITTER.START_FILL (routine)**Symbol:** `blitter_start_fill`**Summary:** Low-level blitter fill start. Call `BLITTER.FILL` for the public wrapper.**Outputs:**

- A: 0 on success, 1 on error.

Source: `blitter.s` line 57**BLITTER.WAIT** (routine)**Symbol:** `blitter_wait`**Summary:** Wait for the blitter controller to leave `BUSY` and return shared status.**Outputs:**

- A: 0 on success, 1 on error.

Source: `blitter.s` line 69

4.12 Expansion RAM

*Generated from `xram.inc`, `xram.s`.***XRAM.ADDRH** (zp)**Symbol:** `XRAM_ADDRH`**Summary:** High byte of flat 24-bit XRAM address.**Value:** `NVR1L` (\$22)**Source:** `xram.inc` line 31**XRAM.ADDRL** (zp)**Symbol:** `XRAM_ADDRL`**Summary:** Low byte of flat 24-bit XRAM address.**Value:** `NVR0L` (\$20)**Source:** `xram.inc` line 21**XRAM.ADDRM** (zp)**Symbol:** `XRAM_ADDRM`**Summary:** Middle byte of flat 24-bit XRAM address.**Value:** `NVR0H` (\$21)**Source:** `xram.inc` line 26**XRAM.ADDR_IN_RANGE** (routine)**Symbol:** `xram_addr_is_in_range`**Summary:** Validate `XRAM_ADDRH` is inside the 512KB XRAM device.**Requires:** `XRAM_ADDRH`**Source:** `xram.s` line 215

XRAM.COPY_FROM_RAM (routine)**Symbol:** xram_copy_from_ram**Summary:** Copy CPU RAM to XRAM using the XRAM pseudo-register ABI.**Requires:** XRAM_RAML, XRAM_RAMH, XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH, XRAM_LEN_L, XRAM_LEN_H**Source:** xram.s line 285**XRAM.COPY_TO_RAM** (routine)**Symbol:** xram_copy_to_ram**Summary:** Copy XRAM to CPU RAM using the XRAM pseudo-register ABI.**Requires:** XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH, XRAM_RAML, XRAM_RAMH, XRAM_LEN_L, XRAM_LEN_H**Source:** xram.s line 312**XRAM.DATA** (zp)**Symbol:** XRAM_DATA**Summary:** Byte value used by XRAM read, write, and fill routines.**Value:** NVR1H (\$23)**Source:** xram.inc line 36**XRAM.FILL** (routine)**Symbol:** xram_fill**Summary:** Fill XRAM using XRAM_ADDRL/M/H, XRAM_LEN_L/H, and XRAM_DATA.**Requires:** XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH, XRAM_LEN_L, XRAM_LEN_H, XRAM_DATA**Source:** xram.s line 339**XRAM.LENH** (zp)**Symbol:** XRAM_LEN_H**Summary:** High byte of XRAM transfer length.**Value:** NVR3H (\$27)**Source:** xram.inc line 56**XRAM.LENL** (zp)**Symbol:** XRAM_LEN_L**Summary:** Low byte of XRAM transfer length.**Value:** NVR3L (\$26)**Source:** xram.inc line 51**XRAM.MAP_WINDOW3** (routine)**Symbol:** xram_map_window3**Summary:** Map XRAM window 3 to XRAM_ADDRM/H.**Requires:** XRAM_ADDRM, XRAM_ADDRH**Source:** xram.s line 233

XRAM.NAMELEN (zp)**Symbol:** XRAM_NAMELEN**Summary:** Filename length for XRAM file streaming routines.**Value:** NVR4L (\$28)**Source:** xram.inc line 61**XRAM.NAMEPTR_H** (zp)**Symbol:** XRAM_NAMEPTR_H**Summary:** High byte of filename pointer for XRAM file streaming routines.**Value:** NVR5L (\$2A)**Source:** xram.inc line 71**XRAM.NAMEPTR_L** (zp)**Symbol:** XRAM_NAMEPTR_L**Summary:** Low byte of filename pointer for XRAM file streaming routines.**Value:** NVR4H (\$29)**Source:** xram.inc line 66**XRAM.RAMH** (zp)**Symbol:** XRAM_RAMH**Summary:** High byte of CPU RAM address for XRAM copy routines.**Value:** NVR2H (\$25)**Source:** xram.inc line 46**XRAM.RAML** (zp)**Symbol:** XRAM_RAML**Summary:** Low byte of CPU RAM address for XRAM copy routines.**Value:** NVR2L (\$24)**Source:** xram.inc line 41**XRAM.READ8** (routine)**Symbol:** xram_read8**Summary:** Read one byte from XRAM_ADDRL/M/H into XRAM_DATA.**Outputs:**

- XRAM_DATA: Byte read from XRAM.

Requires: XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH**Source:** xram.s line 249**XRAM.SET_ERROR** (routine)**Symbol:** xram_set_error**Summary:** Set XMC-compatible XRAM error status.**Inputs:**

- A: XMC-compatible error code.

Outputs:

- A: 1.

Source: xram.s line 52

XRAM.SET_OK (routine)

Symbol: xram_set_ok

Summary: Set XMC-compatible XRAM status to OK and return A=0.

Source: xram.s line 40

XRAM.WAIT_DMA (routine)

Symbol: xram_wait_dma

Summary: Wait for the shared DMA controller and map the result to XRAM status.

Source: xram.s line 374

XRAM.WRITE8 (routine)

Symbol: xram_write8

Summary: Write XRAM_DATA to XRAM_ADDRL/M/H.

Requires: XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH, XRAM_DATA

Source: xram.s line 266

XRAM.XLOAD (routine)

Symbol: xram_xload

Summary: Stream a file directly into XRAM using the XRAM pseudo-register ABI.

Requires: XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH, XRAM_LENH, XRAM_NAMEPTR_L, XRAM_NAMEPTR_H

Source: xram.s line 164

XRAM.XMC_COPY_FROM_RAM (routine)

Symbol: xram_xmc_copy_from_ram

Summary: Copy RAM to XRAM using the XMC register ABI.

Requires: XMC_RAML, XMC_RAMH, XMC_XAL, XMC_XAM, XMC_XAH, XMC_LENH, XMC_LENH

Source: xram.s line 119

XRAM.XMC_COPY_TO_RAM (routine)

Symbol: xram_xmc_copy_to_ram

Summary: Copy XRAM to RAM using the XMC register ABI.

Requires: XMC_XAL, XMC_XAM, XMC_XAH, XMC_RAML, XMC_RAMH, XMC_LENH, XMC_LENH

Source: xram.s line 130

XRAM.XMC_FILL (routine)

Symbol: xram_xmc_fill

Summary: Fill XRAM using the XMC register ABI.

Requires: XMC_XAL, XMC_XAM, XMC_XAH, XMC_LENH, XMC_LENH, XMC_DATA

Source: xram.s line 141

XRAM.XMC_READ8 (routine)**Symbol:** `xram_xmc_read8`**Summary:** Read one byte from XMC_XAL/M/H into XMC_DATA.**Requires:** XMC_XAL, XMC_XAM, XMC_XAH**Source:** `xram.s` line 93**XRAM.XMC_WRITE8** (routine)**Symbol:** `xram_xmc_write8`**Summary:** Write XMC_DATA to XMC_XAL/M/H.**Requires:** XMC_XAL, XMC_XAM, XMC_XAH, XMC_DATA**Source:** `xram.s` line 108**XRAM.XSAVE** (routine)**Symbol:** `xram_xsave`**Summary:** Stream XRAM directly to a file using the XRAM pseudo-register ABI.**Requires:** XRAM_ADDRL, XRAM_ADDRM, XRAM_ADDRH, XRAM_LEN_L, XRAM_LEN_H,
XRAM_NAMELEN, XRAM_NAMEPTR_L, XRAM_NAMEPTR_H**Source:** `xram.s` line 180

4.13 Expansion Memory Controller

*Generated from `xmc.s`.***XMC.ALLOC** (routine)**Symbol:** `xmc_alloc_block`**Summary:** Allocate an XRAM block using XMC_LEN_L/H.**Outputs:**

- XMC_XAL: Allocated address low byte.
- XMC_XAM: Allocated address middle byte.
- XMC_XAH: Allocated address high byte.

Requires: XMC_LEN_L, XMC_LEN_H**Source:** `xmc.s` line 267**XMC.COMMAND_STATUS** (routine)**Symbol:** `xmc_command_status`**Summary:** Process XMC_CMD and return A=0 on XMC_OK, A=1 otherwise.**Outputs:**

- A: 0 on success, 1 on error.

Requires: XMC_CMD**Source:** `xmc.s` line 188

XMC.DIR_OPEN (routine)**Symbol:** `xmc_dir_open`**Summary:** Open the named XRAM block directory.**Source:** `xmc.s` line 306**XMC.DIR_READ** (routine)**Symbol:** `xmc_dir_read`**Summary:** Read the next named XRAM block directory entry.**Source:** `xmc.s` line 314**XMC.FETCH_RAW** (routine)**Symbol:** `xmc_fetch_raw`**Summary:** Copy XRAM to RAM using `XMC_XAL/M/H`, `XMC_RAML/H`, and `XMC_LENH/H`.**Requires:** `XMC_XAL`, `XMC_XAM`, `XMC_XAH`, `XMC_RAML`, `XMC_RAMH`, `XMC_LENH`, `XMC_LENH`**Source:** `xmc.s` line 241**XMC.MAP_WINDOW** (routine)**Symbol:** `xmc_map_window`**Summary:** Map window X to `XMC_XAL/M` in the selected XMC bank.**Inputs:**

- X: Window index, 0..3.

Requires: `XMC_XAL`, `XMC_XAM`, `XMC_BANK`**Source:** `xmc.s` line 390**XMC.NAMED_DELETE** (routine)**Symbol:** `xmc_named_delete`**Summary:** Delete a named XRAM block.**Requires:** `XMC_NAME`, `XMC_NAMELEN`**Source:** `xmc.s` line 297**XMC.NAMED_FETCH** (routine)**Symbol:** `xmc_named_fetch`**Summary:** Fetch a named XRAM block into RAM.**Requires:** `XMC_NAME`, `XMC_NAMELEN`, `XMC_RAML`, `XMC_RAMH`**Source:** `xmc.s` line 288**XMC.NAMED_STASH** (routine)**Symbol:** `xmc_named_stash`**Summary:** Allocate/update a named XRAM block and copy RAM into it.**Requires:** `XMC_NAME`, `XMC_NAMELEN`, `XMC_RAML`, `XMC_RAMH`, `XMC_LENH`, `XMC_LENH`**Source:** `xmc.s` line 279

XMC.PROCESS (routine)**Symbol:** `xmc_process`**Summary:** Process XMC_CMD and update XMC_STATUS/XMC_ERRCODE.**Requires:** XMC_CMD**Source:** `xmc.s` line 53**XMC.PUT_BYTE** (routine)**Symbol:** `xmc_put_byte`**Summary:** Write XMC_DATA to XMC_XAL/M/H.**Requires:** XMC_XAL, XMC_XAM, XMC_XAH, XMC_DATA**Source:** `xmc.s` line 223**XMC.RELEASE** (routine)**Symbol:** `xmc_release`**Summary:** Release an XMC allocation range.**Requires:** XMC_XAL, XMC_XAM, XMC_XAH, XMC_LENH, XMC_LENH**Source:** `xmc.s` line 250**XMC.RESET_USAGE** (routine)**Symbol:** `xmc_reset_usage`**Summary:** Reset XRAM usage and named allocation metadata.**Source:** `xmc.s` line 259**XMC.SELECT_BANK** (routine)**Symbol:** `xmc_select_bank`**Summary:** Select the current 64KB XRAM bank.**Inputs:**

- X: Bank index, 0..XMC_BANKS-1.

Outputs:

- A: 0 on success, 1 if the bank is out of range.

Source: `xmc.s` line 209**XMC.SET_WINDOW_ADDR** (routine)**Symbol:** `xmc_set_window_addr_current_bank`**Summary:** Set a mapped window base from XMC_XAL/M and the selected XMC bank.**Inputs:**

- X: Window index, 0..3.

Requires: XMC_XAL, XMC_XAM, XMC_BANK**Source:** `xmc.s` line 345

XMC.STASH_RAW (routine)**Symbol:** `xmc_stash_raw`**Summary:** Copy RAM to XRAM using `XMC_RAML/H`, `XMC_XAL/M/H`, and `XMC_LENH/H`.**Requires:** `XMC_RAML`, `XMC_RAMH`, `XMC_XAL`, `XMC_XAM`, `XMC_XAH`, `XMC_LENH`, `XMC_LENH`**Source:** `xmc.s` line 232**XMC.UNMAP_WINDOW** (routine)**Symbol:** `xmc_unmap_window`**Summary:** Disable a mapped XRAM window.**Inputs:**

- X: Window index, 0..3.

Source: `xmc.s` line 408**XMC.WINDOW_BIT** (routine)**Symbol:** `xmc_window_bit`**Summary:** Convert window index 0..3 to its `XMC_WINCTL` bit mask.**Inputs:**

- X: Window index, 0..3.

Outputs:

- A: Window bit mask.
- C: Clear on success, set on invalid index.

Source: `xmc.s` line 322

4.14 File Pager

Generated from *pager.inc*, *pager.s*.

PAGER.ADDRH (zp)**Symbol:** `PAGER_ADDRH`**Summary:** High byte of destination address for 24-bit targets.**Value:** `FIO_ARG_ADDRH` (\$22)**Source:** `pager.inc` line 23**PAGER.ADDRL** (zp)**Symbol:** `PAGER_ADDRL`**Summary:** Low byte of destination address.**Value:** `FIO_ARG_ADDRL` (\$20)**Source:** `pager.inc` line 13

PAGER.ADDRM (zp)**Symbol:** PAGER_ADDRM**Summary:** Middle byte of destination address or VGC space selector.**Value:** FIO_ARG_ADDRM (\$21)**Source:** pager.inc line 18**PAGER.FILEH (zp)****Symbol:** PAGER_FILEH**Summary:** High byte of the source file offset.**Value:** NVR6H (\$2D)**Source:** pager.inc line 65**PAGER.FILEL (zp)****Symbol:** PAGER_FILEL**Summary:** Low byte of the source file offset.**Value:** NVR5H (\$2B)**Source:** pager.inc line 55**PAGER.FILEM (zp)****Symbol:** PAGER_FILEM**Summary:** Middle byte of the source file offset.**Value:** NVR6L (\$2C)**Source:** pager.inc line 60**PAGER.LENH (zp)****Symbol:** PAGER_LENH**Summary:** High byte of transfer length.**Value:** FIO_ARG_LENH (\$27)**Source:** pager.inc line 33**PAGER.LENL (zp)****Symbol:** PAGER_LENL**Summary:** Low byte of transfer length.**Value:** FIO_ARG_LENL (\$26)**Source:** pager.inc line 28**PAGER.LOAD_FILE_PAGE (routine)****Symbol:** pager_load_file_page**Summary:** Load PAGER_LEN bytes from PAGER_FILE offset in PAGER_NAME into PAGER_TARGET/PAGER_ADDR.**Outputs:**

- A: 0 on success, 1 on error.

Requires: PAGER_FILEL, PAGER_FILEM, PAGER_FILEH, PAGER_TARGET, PAGER_ADDRL, PAGER_ADDRM, PAGER_ADDRH, PAGER_LENL, PAGER_LENH, PAGER_NAMELEN,

PAGER_NAMEPTR_L, PAGER_NAMEPTR_H

Source: pager.s line 18

PAGER.NAMELEN (zp)

Symbol: PAGER_NAMELEN

Summary: Filename length for the paged file.

Value: FIO_ARG_NAMELEN (\$28)

Source: pager.inc line 38

PAGER.NAMEPTR_H (zp)

Symbol: PAGER_NAMEPTR_H

Summary: High byte of paged filename pointer.

Value: FIO_ARG_NAMEPTR_H (\$2A)

Source: pager.inc line 48

PAGER.NAMEPTR_L (zp)

Symbol: PAGER_NAMEPTR_L

Summary: Low byte of paged filename pointer.

Value: FIO_ARG_NAMEPTR_L (\$29)

Source: pager.inc line 43

PAGER.TARGET (zp)

Symbol: PAGER_TARGET

Summary: Destination target selector: 0=XRAM, 1=CPU RAM, 2=VGC memory.

Value: NVR7L (\$2E)

Source: pager.inc line 72

PAGER.TARGET_RAM (const)

Symbol: PAGER_TARGET_RAM

Summary: PAGER.TARGET value for CPU RAM destinations.

Value: \$01 (\$01)

Source: pager.inc line 83

PAGER.TARGET_VGC (const)

Symbol: PAGER_TARGET_VGC

Summary: PAGER.TARGET value for VGC memory destinations.

Value: \$02 (\$02)

Source: pager.inc line 88

PAGER.TARGET_XRAM (const)

Symbol: PAGER_TARGET_XRAM

Summary: PAGER.TARGET value for flat XRAM destinations.

Value: \$00 (\$00)

Source: pager.inc line 78

4.15 Random Numbers

Generated from *rng.inc*, *rng.s*.

RNG.GET16 (routine)

Symbol: `rng_get16`

Summary: Fill RNG.VALUE0..1 with 16 random bits from the Nova host.

Outputs:

- A: 0 on success, 1 on error.
- RNG_VALUE0: RNG_VALUE1

Source: `rng.s` line 31

RNG.GET32 (routine)

Symbol: `rng_get32`

Summary: Fill RNG.VALUE0..3 with 32 random bits from the Nova host.

Outputs:

- A: 0 on success, 1 on error.
- RNG_VALUE0: RNG_VALUE1 RNG_VALUE2 RNG_VALUE3

Source: `rng.s` line 20

RNG.GET8 (routine)

Symbol: `rng_get8`

Summary: Fill RNG.VALUE0 with an 8-bit random value from the Nova host.

Outputs:

- A: 0 on success, 1 on error.
- RNG_VALUE0:

Source: `rng.s` line 41

RNG.VALUE0 (u8)

Symbol: `RNG_VALUE0`

Summary: Low byte of the most recent RNG.GET32 result.

Value: `FIO_RNG0` (\$B9A4)

Source: `rng.inc` line 12

RNG.VALUE1 (u8)

Symbol: `RNG_VALUE1`

Summary: Second byte of the most recent RNG.GET32 result.

Value: `FIO_RNG1` (\$B9A5)

Source: `rng.inc` line 17

RNG.VALUE2 (u8)**Symbol:** RNG_VALUE2**Summary:** Third byte of the most recent RNG.GET32 result.**Value:** FIO_RNG2 (\$B9A6)**Source:** rng.inc line 22**RNG.VALUE3** (u8)**Symbol:** RNG_VALUE3**Summary:** High byte of the most recent RNG.GET32 result.**Value:** FIO_RNG3 (\$B9A7)**Source:** rng.inc line 27

4.16 Nova Graphics Images

*Generated from nvg.inc, nvg.s.***NVG.ADDRH** (u8)**Symbol:** NVG_ADDRH**Summary:** High byte of the graphics bitmap destination offset for NVG.LOAD_AT.**Value:** FIO_GADDRH (\$B9AC)**Source:** nvg.inc line 17**NVG.ADDRL** (u8)**Symbol:** NVG_ADDRL**Summary:** Low byte of the graphics bitmap destination offset for NVG.LOAD_AT.**Value:** FIO_GADDRL (\$B9AB)**Source:** nvg.inc line 12**NVG.LOAD** (routine)**Symbol:** nvg_load**Summary:** Clear graphics bitmap memory, then decode FIO.NAME as an NVG1 file into graphics bitmap offset 0.**Outputs:**

- A: 0 on success, 1 on error.

Requires: FIO_NAME, FIO_NAMELEN**Source:** nvg.s line 21**NVG.LOAD_AT** (routine)**Symbol:** nvg_load_at**Summary:** Clear graphics bitmap memory, then decode FIO.NAME as an NVG1 file into graphics bitmap offset NVG.ADDRL/H.**Outputs:**

- A: 0 on success, 1 on error.

Requires: FIO_NAME, FIO_NAMELEN, NVG_ADDRL, NVG_ADDRH

Source: `nvg.s` line 35

NVG.LOAD_NAMED (routine)

Symbol: `nvg_load_named`

Summary: Copy NVG.NAMEPTR/NVG.NAMELEN into FIO.NAME, then clear graphics bitmap memory and decode the NVG1 file at offset 0.

Outputs:

- A: 0 on success, 1 on error.

Requires: NVG_NAMELEN, NVG_NAMEPTR_L, NVG_NAMEPTR_H

Source: `nvg.s` line 51

NVG.LOAD_NAMED_AT (routine)

Symbol: `nvg_load_named_at`

Summary: Copy NVG.NAMEPTR/NVG.NAMELEN into FIO.NAME, then clear graphics bitmap memory and decode the NVG1 file at NVG.ADDRL/H.

Outputs:

- A: 0 on success, 1 on error.

Requires: NVG_NAMELEN, NVG_NAMEPTR_L, NVG_NAMEPTR_H, NVG_ADDRL, NVG_ADDRH

Source: `nvg.s` line 65

NVG.NAMELEN (zp)

Symbol: `NVG_NAMELEN`

Summary: Filename length argument for NVG.LOAD_NAMED.

Value: `FIO_ARG_NAMELEN` (\$28)

Source: `nvg.inc` line 22

NVG.NAMEPTR_H (zp)

Symbol: `NVG_NAMEPTR_H`

Summary: High byte of filename pointer argument for NVG.LOAD_NAMED.

Value: `FIO_ARG_NAMEPTR_H` (\$2A)

Source: `nvg.inc` line 32

NVG.NAMEPTR_L (zp)

Symbol: `NVG_NAMEPTR_L`

Summary: Low byte of filename pointer argument for NVG.LOAD_NAMED.

Value: `FIO_ARG_NAMEPTR_L` (\$29)

Source: `nvg.inc` line 27

4.17 Animation

Generated from *anim.inc*, *anim.s*.

ANIM.COUNT (u8)**Symbol:** ANIM_COUNT**Summary:** Number of 128-byte shape slots to load.**Source:** anim.inc line 79**ANIM.DESC_H** (u8)**Symbol:** ANIM_DESC_H**Summary:** High byte of the animation descriptor pointer used by ANIM.START.**Source:** anim.inc line 52**ANIM.DESC_L** (u8)**Symbol:** ANIM_DESC_L**Summary:** Low byte of the animation descriptor pointer used by ANIM.START.**Source:** anim.inc line 47**ANIM.FILEH** (u8)**Symbol:** ANIM_FILEH**Summary:** High byte of source file offset for ANIM.LOAD_DISK_SHAPES.**Source:** anim.inc line 109**ANIM.FILEL** (u8)**Symbol:** ANIM_FILEL**Summary:** Low byte of source file offset for ANIM.LOAD_DISK_SHAPES.**Source:** anim.inc line 99**ANIM.FILEM** (u8)**Symbol:** ANIM_FILEM**Summary:** Middle byte of source file offset for ANIM.LOAD_DISK_SHAPES.**Source:** anim.inc line 104**ANIM.INIT** (routine)**Symbol:** anim_init**Summary:** Clear the animation track table.**Outputs:**

- A: 0 on success.

Source: anim.s line 70**ANIM.LOAD_DISK_SHAPES** (routine)**Symbol:** anim_load_disk_shapes**Summary:** Stream ANIM.COUNT shape slots from a file slice directly into VGC sprite shape RAM at ANIM.SHAPE.**Outputs:**

- A: 0 on success, 1 on error.

Requires: ANIM_NAMEPTR_L, ANIM_NAMEPTR_H, ANIM_NAMELEN, ANIM_FILEL, ANIM_FILEM, ANIM_FILEH, ANIM_SHAPE, ANIM_COUNT

Source: anim.s line 278

ANIM.LOAD_XRAM_SHAPES (routine)

Symbol: anim_load_xram_shapes

Summary: DMA ANIM.COUNT shape slots from ANIM.XADDR into VGC sprite shape RAM at ANIM.SHAPE.

Outputs:

- A: 0 on success, 1 on error.

Requires: ANIM_XADDRL, ANIM_XADDRM, ANIM_XADDRH, ANIM_SHAPE, ANIM_COUNT

Source: anim.s line 236

ANIM.NAMELEN (u8)

Symbol: ANIM_NAMELEN

Summary: Filename length for ANIM.LOAD_DISK_SHAPES.

Source: anim.inc line 114

ANIM.NAMEPTR_H (u8)

Symbol: ANIM_NAMEPTR_H

Summary: High byte of filename pointer for ANIM.LOAD_DISK_SHAPES.

Source: anim.inc line 124

ANIM.NAMEPTR_L (u8)

Symbol: ANIM_NAMEPTR_L

Summary: Low byte of filename pointer for ANIM.LOAD_DISK_SHAPES.

Source: anim.inc line 119

ANIM.RESULT (u8)

Symbol: ANIM_RESULT

Summary: Result code from the last animation operation.

Source: anim.inc line 67

ANIM.SET_FRAME (routine)

Symbol: anim_set_frame

Summary: Set one animation track to a frame index and apply it immediately.

Inputs:

- A: Track handle.
- X: Frame index.

Outputs:

- A: Result code.

Source: anim.s line 204

ANIM.SHAPE (u8)

Symbol: ANIM_SHAPE

Summary: Destination shape slot for bulk shape loading.

Source: anim.inc line 74

ANIM.START (routine)

Symbol: anim_start

Summary: Attach ANIM.DESC to ANIM.TARGET and return a track handle.

Outputs:

- A: Track handle on success, \$FF on failure.

Requires: ANIM_DESC_L, ANIM_DESC_H, ANIM_TARGET, ANIM_TARGET_TYPE

Source: anim.s line 89

ANIM.STOP (routine)

Symbol: anim_stop

Summary: Stop and release one animation track.

Inputs:

- A: Track handle.

Outputs:

- A: Result code.

Source: anim.s line 150

ANIM.TARGET (u8)

Symbol: ANIM_TARGET

Summary: Hardware sprite index or meta-sprite handle used by ANIM.START.

Source: anim.inc line 57

ANIM.TARGET_TYPE (u8)

Symbol: ANIM_TARGET_TYPE

Summary: Target type selector: ANIM_TARGET_SPRITE or ANIM_TARGET_MSPRITE.

Source: anim.inc line 62

ANIM.TICK (routine)

Symbol: anim_tick

Summary: Advance every running animation track and apply changed frames.

Outputs:

- A: 0 on success.

Source: anim.s line 164

ANIM.TICK_ONE (routine)**Symbol:** `anim_tick_one`**Summary:** Advance one running animation track.**Inputs:**

- A: Track handle.

Outputs:

- A: Result code.

Source: `anim.s` line 185**ANIM.XADDRH** (u8)**Symbol:** `ANIM_XADDRH`**Summary:** High byte of flat XRAM source address for `ANIM.LOAD_XRAM_SHAPES`.**Source:** `anim.inc` line 94**ANIM.XADDRL** (u8)**Symbol:** `ANIM_XADDRL`**Summary:** Low byte of flat XRAM source address for `ANIM.LOAD_XRAM_SHAPES`.**Source:** `anim.inc` line 84**ANIM.XADDRM** (u8)**Symbol:** `ANIM_XADDRM`**Summary:** Middle byte of flat XRAM source address for `ANIM.LOAD_XRAM_SHAPES`.**Source:** `anim.inc` line 89

4.18 Tweening

Generated from `tween.inc`, `tween.s`.

TWEEN.BEGIN (routine)**Symbol:** `tween_begin`**Summary:** Reset `TWEEN.FRAME/DONE` and set `TWEEN.VALUE` to `TWEEN.START`.**Outputs:**

- `TWEEN_FRAME`: Reset to 0.
- `TWEEN_DONE`: Cleared to 0.
- `TWEEN_VALUEL`: Low byte of `TWEEN.START`.
- `TWEEN_VALUEH`: High byte of `TWEEN.START`.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`**Source:** `tween.s` line 52

TWEEN.DONE (u8)**Symbol:** TWEEN_DONE**Summary:** Non-zero when the tween has reached its end value.**Source:** tween.inc line 81**TWEEN.DURATION** (u8)**Symbol:** TWEEN_DURATION**Summary:** Tween length in frames. Zero snaps to the end value.**Source:** tween.inc line 56**TWEEN.EASE** (u8)**Symbol:** TWEEN_EASE**Summary:** Last eased progress value, 0..255.**Source:** tween.inc line 91**TWEEN.ENDH** (u8)**Symbol:** TWEEN_ENDH**Summary:** High byte of the tween end value.**Source:** tween.inc line 51**TWEEN.ENDL** (u8)**Symbol:** TWEEN_ENDL**Summary:** Low byte of the tween end value.**Source:** tween.inc line 46**TWEEN.EVAL** (routine)**Symbol:** tween_eval**Summary:** Evaluate the current tween frame using TWEEN.MODE.**Outputs:**

- TWEEN_VALUEL: Low byte of computed value.
- TWEEN_VALUEH: High byte of computed value.
- TWEEN_DONE: Non-zero when the end value has been reached.
- TWEEN_PROGRESS: Normalized progress, 0..255.
- TWEEN_EASE: Eased progress, 0..255.

Requires: TWEEN_STARTL, TWEEN_STARTH, TWEEN_ENDL, TWEEN_ENDH, TWEEN_DURATION, TWEEN_FRAME, TWEEN_MODE**Source:** tween.s line 71**TWEEN.EVAL_EASE_IN** (routine)**Symbol:** tween_eval_ease_in**Summary:** Evaluate the current tween frame with slow-start dampening.

Outputs:

- `TWEEN_VALUEL`: Low byte of computed value.
- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`

Source: `tween.s` line 121

`TWEEN.EVAL_EASE_IN_OUT` (routine)

Symbol: `tween_eval_ease_in_out`

Summary: Evaluate the current tween frame with dampening at both ends.

Outputs:

- `TWEEN_VALUEL`: Low byte of computed value.
- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`

Source: `tween.s` line 180

`TWEEN.EVAL_EASE_OUT` (routine)

Symbol: `tween_eval_ease_out`

Summary: Evaluate the current tween frame with slow-end dampening.

Outputs:

- `TWEEN_VALUEL`: Low byte of computed value.
- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`

Source: `tween.s` line 145

TWEEN.EVAL_LINEAR (routine)**Symbol:** `tween_eval_linear`**Summary:** Evaluate the current tween frame with a linear curve.**Outputs:**

- `TWEEN_VALUEL`: Low byte of computed value.
- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`**Source:** `tween.s` line 98**TWEEN.FRAME** (u8)**Symbol:** `TWEEN_FRAME`**Summary:** Current tween frame, from 0 through `TWEEN.DURATION`.**Source:** `tween.inc` line 61**TWEEN.MODE** (u8)**Symbol:** `TWEEN_MODE`**Summary:** Easing mode: 0 linear, 1 ease-in, 2 ease-out, 3 ease-in-out.**Source:** `tween.inc` line 66**TWEEN.MODE_EASE_IN** (const)**Symbol:** `TWEEN_MODE_EASE_IN`**Summary:** Constant for quadratic ease-in interpolation.**Value:** 1 (\$01)**Source:** `tween.inc` line 20**TWEEN.MODE_EASE_IN_OUT** (const)**Symbol:** `TWEEN_MODE_EASE_IN_OUT`**Summary:** Constant for quadratic ease-in/ease-out interpolation.**Value:** 3 (\$03)**Source:** `tween.inc` line 30**TWEEN.MODE_EASE_OUT** (const)**Symbol:** `TWEEN_MODE_EASE_OUT`**Summary:** Constant for quadratic ease-out interpolation.**Value:** 2 (\$02)**Source:** `tween.inc` line 25

TWEEN.MODE_LINEAR (const)**Symbol:** TWEEN_MODE_LINEAR**Summary:** Constant for linear interpolation.**Value:** 0 (\$00)**Source:** tween.inc line 15**TWEEN.PROGRESS** (u8)**Symbol:** TWEEN_PROGRESS**Summary:** Last normalized progress value, 0..255.**Source:** tween.inc line 86**TWEEN.STARTH** (u8)**Symbol:** TWEEN_STARTH**Summary:** High byte of the tween start value.**Source:** tween.inc line 41**TWEEN.STARTL** (u8)**Symbol:** TWEEN_STARTL**Summary:** Low byte of the tween start value.**Source:** tween.inc line 36**TWEEN.STEP** (routine)**Symbol:** tween_step**Summary:** Increment TWEEN.FRAME by one, then evaluate using TWEEN.MODE.**Outputs:**

- TWEEN_FRAME: Advanced frame value.
- TWEEN_VALUEL: Low byte of computed value.
- TWEEN_VALUEH: High byte of computed value.
- TWEEN_DONE: Non-zero when the end value has been reached.
- TWEEN_PROGRESS: Normalized progress, 0..255.
- TWEEN_EASE: Eased progress, 0..255.

Requires: TWEEN_STARTL, TWEEN_STARTH, TWEEN_ENDL, TWEEN_ENDH, TWEEN_DURATION, TWEEN_FRAME, TWEEN_MODE**Source:** tween.s line 217**TWEEN.STEP_EASE_IN** (routine)**Symbol:** tween_step_ease_in**Summary:** Increment TWEEN.FRAME by one, then evaluate with slow-start dampening.**Outputs:**

- TWEEN_FRAME: Advanced frame value.
- TWEEN_VALUEL: Low byte of computed value.

- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`

Source: `tween.s` line 250

`TWEEN.STEP_EASE_IN_OUT` (routine)

Symbol: `tween_step_ease_in_out`

Summary: Increment `TWEEN.FRAME` by one, then evaluate with dampening at both ends.

Outputs:

- `TWEEN_FRAME`: Advanced frame value.
- `TWEEN_VALUEL`: Low byte of computed value.
- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`

Source: `tween.s` line 284

`TWEEN.STEP_EASE_OUT` (routine)

Symbol: `tween_step_ease_out`

Summary: Increment `TWEEN.FRAME` by one, then evaluate with slow-end dampening.

Outputs:

- `TWEEN_FRAME`: Advanced frame value.
- `TWEEN_VALUEL`: Low byte of computed value.
- `TWEEN_VALUEH`: High byte of computed value.
- `TWEEN_DONE`: Non-zero when the end value has been reached.
- `TWEEN_PROGRESS`: Normalized progress, 0..255.
- `TWEEN_EASE`: Eased progress, 0..255.

Requires: `TWEEN_STARTL`, `TWEEN_STARTH`, `TWEEN_ENDL`, `TWEEN_ENDH`, `TWEEN_DURATION`, `TWEEN_FRAME`

Source: `tween.s` line 267

TWEEN.STEP_LINEAR (routine)**Symbol:** `tween_step_linear`**Summary:** Increment TWEEN.FRAME by one, then evaluate with a linear curve.**Outputs:**

- TWEEN_FRAME: Advanced frame value.
- TWEEN_VALUEL: Low byte of computed value.
- TWEEN_VALUEH: High byte of computed value.
- TWEEN_DONE: Non-zero when the end value has been reached.
- TWEEN_PROGRESS: Normalized progress, 0..255.
- TWEEN_EASE: Eased progress, 0..255.

Requires: TWEEN_STARTL, TWEEN_STARTH, TWEEN_ENDL, TWEEN_ENDH, TWEEN_DURATION, TWEEN_FRAME**Source:** `tween.s` line 233**TWEEN.VALUEH** (u8)**Symbol:** `TWEEN_VALUEH`**Summary:** High byte of the last tween result.**Source:** `tween.inc` line 76**TWEEN.VALUEL** (u8)**Symbol:** `TWEEN_VALUEL`**Summary:** Low byte of the last tween result.**Source:** `tween.inc` line 71

4.19 Network Interface

*Generated from `nic.inc`, `nic.s`.***NIC.ACCEPT** (routine)**Symbol:** `nic_accept`**Summary:** Accept a pending connection for NIC_SLOT.**Source:** `nic.s` line 82**NIC.ARG_NAMELEN** (zp)**Symbol:** `NIC_ARG_NAMELEN`**Summary:** Hostname length for NIC.COPY_NAME.**Value:** `NVR0L` (\$20)**Source:** `nic.inc` line 45

NIC.ARG_NAMEPTR_H (zp)**Symbol:** NIC_ARG_NAMEPTR_H**Summary:** High byte of hostname pointer for NIC.COPY_NAME.**Value:** NVR1L (\$22)**Source:** nic.inc line 55**NIC.ARG_NAMEPTR_L** (zp)**Symbol:** NIC_ARG_NAMEPTR_L**Summary:** Low byte of hostname pointer for NIC.COPY_NAME.**Value:** NVR0H (\$21)**Source:** nic.inc line 50**NIC.COMMAND** (routine)**Symbol:** nic_command**Summary:** Issue a raw NIC command.**Inputs:**

- A: NIC command byte.

Source: nic.s line 29**NIC.CONNECT** (routine)**Symbol:** nic_connect**Summary:** Connect NIC_SLOT to NIC_NAMEBUF:NIC_RPORTL/H.**Requires:** NIC_SLOT, NIC_NAMEBUF, NIC_RPORTL, NIC_RPORTH**Source:** nic.s line 55**NIC.COPY_NAME** (routine)**Symbol:** nic_copy_name**Summary:** Copy a pointer-based host name to NIC_NAMEBUF and append NUL.**Outputs:**

- A: 0 on success, 1 on invalid length.

Requires: NIC_ARG_NAMELEN, NIC_ARG_NAMEPTR_L, NIC_ARG_NAMEPTR_H**Source:** nic.s line 149**NIC.DISCONNECT** (routine)**Symbol:** nic_disconnect**Summary:** Disconnect NIC_SLOT.**Requires:** NIC_SLOT**Source:** nic.s line 64**NIC.LENGTH** (routine)**Symbol:** nic_length**Summary:** Read the most recent NIC message length.**Outputs:**

- A: Message length byte.

Source: `nic.s` line 138

NIC.LISTEN (routine)

Symbol: `nic_listen`

Summary: Listen on `NIC_SLOT` using `NIC_LPORTL/H`.

Source: `nic.s` line 74

NIC.READY (routine)

Symbol: `nic_ready`

Summary: Return 1 when `NIC_SLOT` has data/accept readiness, otherwise 0.

Inputs:

- X: Slot number.

Outputs:

- A: 1 when ready, 0 otherwise.

Source: `nic.s` line 124

NIC.RECV (routine)

Symbol: `nic_recv`

Summary: Receive data for `NIC_SLOT` into `NIC_DMAL/H`.

Requires: `NIC_SLOT`, `NIC_DMAL`, `NIC_DMAH`

Source: `nic.s` line 100

NIC.SEND (routine)

Symbol: `nic_send`

Summary: Send `NIC_DMALEN` bytes from `NIC_DMAL/H` on `NIC_SLOT`.

Requires: `NIC_SLOT`, `NIC_DMAL`, `NIC_DMAH`, `NIC_DMALEN`

Source: `nic.s` line 91

NIC.STATUS (routine)

Symbol: `nic_status`

Summary: Read clamped NIC slot status.

Inputs:

- X: Slot number.

Outputs:

- A: Slot status byte.

Source: `nic.s` line 110

4.20 Game Server

Generated from `gameserver.s`.

NGS.BEGIN_MESSAGE (routine)

Symbol: `ngs_begin_message`

Summary: Begin a request frame and allocate the next non-zero sequence byte.

Source: `gameserver.s` line 430

NGS.BUILD_AI_ADD_TO_TABLE (routine)

Symbol: `ngs_build_ai_add_to_table`

Summary: Build `AI_ADD_TO_TABLE` with `NGS_TABLE_IDL/H` and `NGS_VALUE_L/H` provider id.

Source: `gameserver.s` line 1077

NGS.BUILD_AI_LIST_REQUEST (routine)

Symbol: `ngs_build_ai_list_request`

Summary: Build `AI_LIST_REQUEST`, optionally filtered by non-zero `NGS_GAME_IDL/H`.

Source: `gameserver.s` line 1060

NGS.BUILD_GAME_LIST_REQUEST (routine)

Symbol: `ngs_build_game_list_request`

Summary: Build `GAME_LIST_REQUEST`.

Source: `gameserver.s` line 938

NGS.BUILD_HELLO (routine)

Symbol: `ngs_build_hello`

Summary: Build `HELLO` using `NGS_STRPTRL/H:NGS_STRLEN` as the requested handle.

Source: `gameserver.s` line 864

NGS.BUILD_LOBBY_CHAT (routine)

Symbol: `ngs_build_lobby_chat`

Summary: Build `LOBBY_CHAT` from `NGS_STRPTRL/H:NGS_STRLEN`.

Source: `gameserver.s` line 904

NGS.BUILD_OPPONENT_LIST_REQUEST (routine)

Symbol: `ngs_build_opponent_list_request`

Summary: Build `OPPONENT_LIST_REQUEST`, optionally filtered by non-zero `NGS_GAME_IDL/H`.

Source: `gameserver.s` line 1096

NGS.BUILD_PING (routine)

Symbol: `ngs_build_ping`

Summary: Build an empty `PING` request.

Source: `gameserver.s` line 896

NGS.BUILD_PLAYER_ACTION (routine)**Symbol:** `ngs_build_player_action`**Summary:** Build `PLAYER_ACTION` with table id, action type, and raw payload bytes.**Source:** `gameserver.s` line 1028**NGS.BUILD_STATE_REQUEST** (routine)**Symbol:** `ngs_build_state_request`**Summary:** Build `STATE_REQUEST` with `NGS_TABLE_IDL/H`.**Source:** `gameserver.s` line 1049**NGS.BUILD_TABLE_CHAT** (routine)**Symbol:** `ngs_build_table_chat`**Summary:** Build `TABLE_CHAT` with `NGS_TABLE_IDL/H` and string text.**Source:** `gameserver.s` line 913**NGS.BUILD_TABLE_CREATE** (routine)**Symbol:** `ngs_build_table_create`**Summary:** Build `TABLE_CREATE` using game id/version and `NGS_STRPTRL/H` table name.**Source:** `gameserver.s` line 963**NGS.BUILD_TABLE_JOIN** (routine)**Symbol:** `ngs_build_table_join`**Summary:** Build `TABLE_JOIN` with `NGS_TABLE_IDL/H` and `NGS_ROLE`.**Source:** `gameserver.s` line 988**NGS.BUILD_TABLE_LEAVE** (routine)**Symbol:** `ngs_build_table_leave`**Summary:** Build `TABLE_LEAVE` with `NGS_TABLE_IDL/H`.**Source:** `gameserver.s` line 1017**NGS.BUILD_TABLE_LIST_REQUEST** (routine)**Symbol:** `ngs_build_table_list_request`**Summary:** Build `TABLE_LIST_REQUEST`, optionally filtered by non-zero `NGS_GAME_IDL/H`.**Source:** `gameserver.s` line 946**NGS.BUILD_TABLE_OBSERVE** (routine)**Symbol:** `ngs_build_table_observe`**Summary:** Build `TABLE_OBSERVE` with `NGS_TABLE_IDL/H`.**Source:** `gameserver.s` line 1006

NGS.BUILD_USER_LIST_REQUEST (routine)

Symbol: `ngs_build_user_list_request`
Summary: Build USER_LIST_REQUEST.
Source: `gameserver.s` line 930

NGS.CONNECT (routine)

Symbol: `ngs_connect`
Summary: Connect NGS_SLOT to NGS_STRPTRL/H:NGS_STRLEN at NGS_PORTL/H.
Source: `gameserver.s` line 195

NGS.CONNECT_DEFAULT (routine)

Symbol: `ngs_connect_default`
Summary: Connect NGS_SLOT to the default Nova game-server endpoint.
Source: `gameserver.s` line 220

NGS.DISCONNECT (routine)

Symbol: `ngs_disconnect`
Summary: Disconnect NGS_SLOT.
Source: `gameserver.s` line 235

NGS.INIT (routine)

Symbol: `ngs_init`
Summary: Initialize game-server client state and default port 6503.
Source: `gameserver.s` line 118

NGS.KEEPALIVE_RESET (routine)

Symbol: `ngs_keepalive_reset`
Summary: Reset the keepalive countdown to NGS_KEEPALIVE_INTERVAL_L/H.
Source: `gameserver.s` line 281

NGS.KEEPALIVE_SET_INTERVAL (routine)

Symbol: `ngs_keepalive_set_interval`
Summary: Set the client keepalive ping interval used by NGS.KEEPALIVE_TICK.
Source: `gameserver.s` line 272

NGS.KEEPALIVE_TICK (routine)

Symbol: `ngs_keepalive_tick`
Summary: Send periodic PING frames while an app is blocked in UI/input code.
Source: `gameserver.s` line 294

NGS.PARSE_ENVELOPE (routine)**Symbol:** `ngs_parse_envelope`**Summary:** Parse the frame envelope at `NGS_BUFL/H` using `NGS_LAST_LEN`.**Source:** `gameserver.s` line 726**NGS.PARSE_ERROR** (routine)**Symbol:** `ngs_parse_error`**Summary:** Parse current ERROR payload into `NGS_ERROR_CODE` and `NGS_ERROR_TEXT*`.**Source:** `gameserver.s` line 622**NGS.PING_ROUNDTRIP** (routine)**Symbol:** `ngs_ping_roundtrip`**Summary:** Send PING and wait for PONG using the shared timeout/error handling.**Source:** `gameserver.s` line 709**NGS.READY** (routine)**Symbol:** `ngs_ready`**Summary:** Return `A=1` when `NGS_SLOT` has a queued message, otherwise `A=0`.**Source:** `gameserver.s` line 253**NGS.READ_BYTE** (routine)**Symbol:** `ngs_read_byte`**Summary:** Read one byte from the parsed payload into `A`.**Source:** `gameserver.s` line 780**NGS.READ_STRING** (routine)**Symbol:** `ngs_read_string`**Summary:** Read a length-prefixed string pointer into `NGS_STRPTR/L/H:NGS_STRLEN`.**Source:** `gameserver.s` line 825**NGS.READ_U16** (routine)**Symbol:** `ngs_read_u16`**Summary:** Read a little-endian u16 into `NGS_VALUE_L/H` and `A/X`.**Source:** `gameserver.s` line 805**NGS.RECV_CURRENT** (routine)**Symbol:** `ngs_recv_current`**Summary:** Receive and parse one queued frame from `NGS_SLOT` into `NGS_BUFL/H`.**Source:** `gameserver.s` line 596

NGS.SEND_CURRENT (routine)**Symbol:** `ngs_send_current`**Summary:** Send the current frame through NGS_SLOT using NIC DMA registers.**Source:** `gameserver.s` line 567**NGS.SET_BUFFER** (routine)**Symbol:** `ngs_set_buffer`**Summary:** Set the 256-byte CPU RAM frame buffer.**Source:** `gameserver.s` line 175**NGS.SET_STRING** (routine)**Symbol:** `ngs_set_string`**Summary:** Set the pointer/length pair used by host, string, and raw payload routines.**Source:** `gameserver.s` line 185**NGS.SET_TIMEOUT** (routine)**Symbol:** `ngs_set_timeout`**Summary:** Set shared wait timeout in video frames; zero waits indefinitely.**Source:** `gameserver.s` line 262**NGS.STATUS** (routine)**Symbol:** `ngs_status`**Summary:** Return NGS_SLOT status in A.**Source:** `gameserver.s` line 245**NGS.WAIT_CONNECTED** (routine)**Symbol:** `ngs_wait_connected`**Summary:** Wait for NGS_SLOT to connect or fail within NGS_TIMEOUT_L/H frames.**Source:** `gameserver.s` line 389**NGS.WAIT_KIND** (routine)**Symbol:** `ngs_wait_kind`**Summary:** Wait for a non-error frame of the requested kind, skipping unrelated events.**Source:** `gameserver.s` line 692**NGS.WAIT_MESSAGE** (routine)**Symbol:** `ngs_wait_message`**Summary:** Wait for and receive one frame; protocol ERROR frames are parsed and returned as NGS_ERR_PROTOCOL_ERROR.**Source:** `gameserver.s` line 652

NGS.WRITE_BYTE (routine)**Symbol:** `ngs_write_byte`**Summary:** Append one payload byte to the current frame.**Source:** `gameserver.s` line 459**NGS.WRITE_BYTES** (routine)**Symbol:** `ngs_write_bytes`**Summary:** Append raw bytes from `NGS_STRPTRL/H` without a length prefix.**Source:** `gameserver.s` line 536**NGS.WRITE_STRING** (routine)**Symbol:** `ngs_write_string`**Summary:** Append `NGS_STRLEN` plus bytes from `NGS_STRPTRL/H`.**Source:** `gameserver.s` line 516**NGS.WRITE_U16** (routine)**Symbol:** `ngs_write_u16`**Summary:** Append a little-endian 16-bit value.**Source:** `gameserver.s` line 479

4.21 Overlays

*Generated from `overlay.s`.***OVERLAY.CALL_INIT** (routine)**Symbol:** `overlay_call_init`**Summary:** Call the active overlay init entry if one is declared.**Source:** `overlay.s` line 385**OVERLAY.CALL_MAIN** (routine)**Symbol:** `overlay_call_main`**Summary:** Call the active overlay main entry. Missing main is an error.**Source:** `overlay.s` line 395**OVERLAY.CALL_TICK** (routine)**Symbol:** `overlay_call_tick`**Summary:** Call the active overlay tick entry if one is declared.**Source:** `overlay.s` line 405**OVERLAY.CLEAR_ACTIVE** (routine)**Symbol:** `overlay_clear_active`**Summary:** Forget the active overlay metadata without touching the RAM slot.**Source:** `overlay.s` line 76

OVERLAY.LOAD_FIXED (routine)**Symbol:** `overlay_load_fixed`**Summary:** Load and activate a NOVO fixed-address overlay into CPU RAM.**Outputs:**

- A: 0 on success, 1 on error. `OVL_RESULT` holds `OVL_ERR_*` detail.

Requires: `OVL_NAMEPTR_L`, `OVL_NAMEPTR_H`, `OVL_NAMELEN`, `OVL_LOADL`, `OVL_LOADH`, `OVL_MAXLENL`, `OVL_MAXLENH`**Source:** `overlay.s` line 102**OVERLAY.OK** (routine)**Symbol:** `overlay_ok`**Summary:** Store `OVL.OK` and return `A=0`.**Source:** `overlay.s` line 56**OVERLAY.SET_ERROR** (routine)**Symbol:** `overlay_set_error`**Summary:** Store a detailed overlay error code.**Inputs:**

- A: `OVL_ERR_*` code.

Outputs:

- A: 1.

Source: `overlay.s` line 65**OVERLAY.UNLOAD** (routine)**Symbol:** `overlay_unload`**Summary:** Call the active overlay unload entry if present, then clear metadata.**Outputs:**

- A: 0 on success, 1 on unload-entry error.

Source: `overlay.s` line 364

4.22 VGC Wait Helpers

*Generated from `vgc_wait.s`.***VGC.WAIT_COMMAND** (routine)**Symbol:** `vgc_wait_cmd`**Summary:** Wait for the active VGC hardware command to complete.**Source:** `vgc_wait.s` line 16

About This Generated Reference

The library reference in this book is generated from ‘@label’ documentation blocks in ‘runtime/asm’. The generated JSON file is written next to the LaTeX output so other tools can consume the same API description.

Mermaid diagrams can be added under ‘diagrams/’ as ‘.mmd’ files. The book Makefile renders them to SVG under ‘build/diagrams/’; LaTeX can then include the SVG assets through the shared book style.

The generated reference should fail loudly when required structured fields are missing. That keeps the NDK documentation useful for assembly programmers and prevents the book from drifting away from the code.